

Faiyke RMM System

Technical Guide

Architecture, data models, API reference, and extension guide

Version 1.5 · September 2026

Prepared for

Faiyke-AI Agency

RMM System — Technical Guide

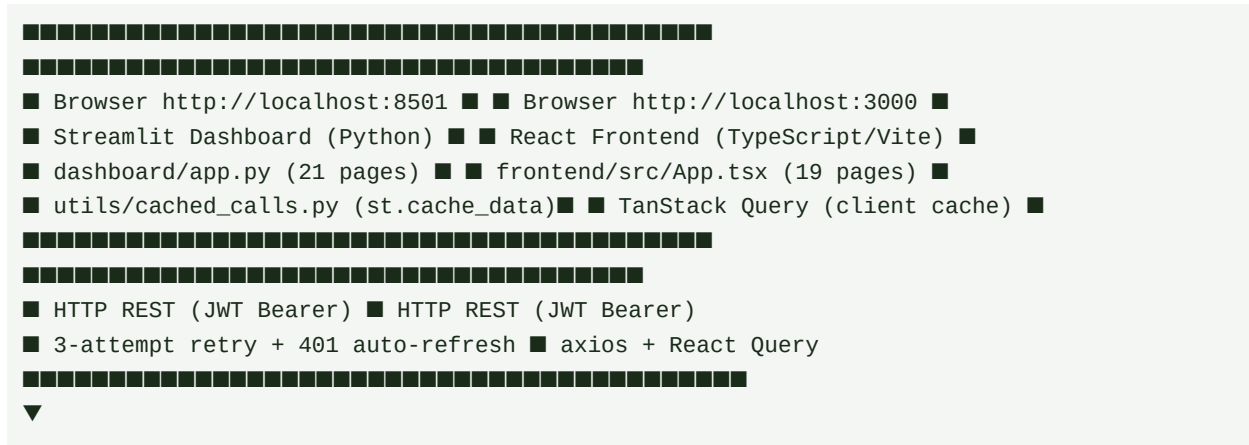
Audience: Developers, system architects, and advanced administrators **Stack:** Flask 3 · SQLAlchemy 2 · Celery 5 · Streamlit 1.58 · React 19/Vite 8/TypeScript · PostgreSQL 15 · Redis/Memurai **Version:** 1.5 (React frontend, cross-platform agent, screenshot pipeline, IoT/MQTT, performance hardening, Android MDM, AI Assistant services refactor, API/token usage monitoring)

Table of Contents

- 1. System Architecture
- 2. Data Models
- 3. API Endpoints
- 4. Authentication & Authorization
- 5. Celery Tasks & Schedules
- 6. Agent Internals
- 7. Built-in Scripts Reference
- 8. Dashboard API Client
- 9. Performance Optimizations
- 10. Extension Guide
- 11. Security Model
- 12. Environment Variables Reference
- 13. Docker Deployment
- 14. AI Assistant
- 15. React Frontend
- 16. Screenshot Capture Pipeline
- 17. IoT / MQTT / SNMP
- 18. Test Suite
- 19. Mobile Device Management
- 20. API & Token Usage Monitoring

1. System Architecture

Component Diagram




```

■■ ScriptRun(script_id=<builtin_reboot_id>, device_id=...)
■■ db.session.add + commit
■
(60s later)
■
▼
agent polls GET /api/agents/<id>/tasks
■■ returns [{type: "run_script", script_content: "..."}]
■■ executor.run(task)
■■ script_runner.run_ps1("Restart-Computer -Force")

```

2. Data Models

Model Relationships

```

Customer ■■< Device ■■< DeviceMetrics
■ ■■■< InstalledSoftware
■ ■■■< AlertRule.customer_id (optional)
■ ■■■< PatchRecord
■ ■■■< ScriptRun
■ ■■■< ScheduledTaskRun
■
■■■< Ticket ■■< TicketComment
■ ■■■ assignee_id → User
■
■■■< PatchPolicy
■■■< AutomationProfile ■■< ScheduledTaskRun
■■■< Invoice
■■■< Report
■■■< NetworkScan

AlertRule ■■< Alert
Script ■■< ScriptRun
User ■■< AuditLog
Device ■■< AgentToken

```

Model Reference

User (users)

```
id Integer PK
email String(255) UNIQUE NOT NULL
password_hash String(255) # bcrypt rounds=12
full_name String(255)
role String(50) # superadmin | admin | technician | viewer
# superadmin: highest privilege, bypasses all _require_role() checks,
# cannot be deleted or modified via API
mfa_secret String(32)
mfa_enabled Boolean default False
is_active Boolean default True
must_change_password Boolean default False server_default="false"
# True → intercept login with force-change screen
created_at DateTime server_default=now()
```

Customer (customers)

```
id Integer PK
name String(255) NOT NULL
slug String(100) UNIQUE
email String(255)
phone String(50)
tier String(50) default 'standard' # standard | premium | enterprise
notes Text
is_active Boolean default True
created_at DateTime
```

Device (devices)

```
id String/UUID PK
customer_id FK → customers
group_id FK → device_groups (nullable)
hostname String(255)
platform String(50) # windows | linux | mac
os_name String(255)
os_version String(100)
cpu_brand String(255)
cpu_cores Integer
ram_gb Float
ip_address String(50) # local IP
serial_number String(255)
hardware_id String(64) UNIQUE # SHA256(hostname+MAC)
status String(50) default 'active'
is_online Boolean default False
last_seen DateTime
agent_version String(20)
display_name String(255)
created_at DateTime
is_agentless Boolean default False NOT NULL # True = network-discovered, no agent
device_type String(50) default 'laptop' NOT NULL # laptop | desktop | mobile | server |
unknown
vendor String(255) nullable # OUI lookup result, e.g. "Apple, Inc."
# NOTE: customer_id is nullable=True – agentless devices start unassigned
```

DeviceMetrics (device_metrics)

```
id Integer PK
device_id FK → devices
cpu_pct Float
ram_pct Float
disk_pct Float # primary disk
battery_pct Float (nullable)
uptime_seconds BigInteger
top_processes JSON # [{pid, name, cpu}]
disks JSON # [{mountpoint, device, total_gb, used_gb, free_gb, percent}]
collected_at DateTime # NOTE: field is collected_at not recorded_at
```

AlertRule (alert_rules)

```
id Integer PK
name String(255) NOT NULL
metric String(100) # cpu_pct | ram_pct | disk_pct | offline
operator String(10) # > | < | ==
threshold Float
severity String(20) # info | warning | critical
cooldown_minutes Integer default 15
is_active Boolean default True
customer_id FK → customers (nullable = global rule)
notification_channels JSON # {"email": ["addr@domain.com"]}
```

Alert (alerts)

```
id Integer PK
rule_id FK → alert_rules
device_id FK → devices
severity String(20)
status String(20) default 'open' # open | acknowledged | resolved
message Text
acknowledged_by FK → users (nullable)
created_at DateTime
resolved_at DateTime (nullable)
```

Ticket + TicketComment

```
# Ticket
id String/UUID PK
title String(500) NOT NULL
description Text
customer_id FK → customers
device_id FK → devices (nullable)
assignee_id FK → users (nullable) # Phase 8 addition
priority String(20) default 'medium' # low | medium | high | critical
status String(20) default 'open' # open | in_progress | resolved | closed
source String(50) # manual | alert | agent
created_at DateTime
updated_at DateTime onupdate=now()

# TicketComment
id Integer PK
ticket_id FK → tickets
author_id FK → users
body Text NOT NULL
is_internal Boolean default False
created_at DateTime
```

Script + ScriptRun

```
# Script
id String/UUID PK
name String(255) NOT NULL
description Text # used as __tag__ for built-in scripts
file_type String(20) # ps1 | bat | py | sh
content Text NOT NULL
os_target String(20) default 'windows'
is_builtin Boolean default False
created_by FK → users (nullable)
created_at DateTime

# ScriptRun
id String/UUID PK
script_id FK → scripts NOT NULL
device_id FK → devices
triggered_by FK → users (nullable)
exit_code Integer
stdout Text
stderr Text
status String(20) default 'pending' # pending | running | completed | failed | timeout
timeout_seconds Integer default 300
started_at DateTime (nullable)
completed_at DateTime (nullable)
created_at DateTime
```

PatchPolicy + PatchRecord

```
# PatchPolicy
id Integer PK
name String(255) NOT NULL
customer_id FK → customers (nullable)
auto_approve_critical Boolean default True
auto_approve_important Boolean default False
auto_approve_security Boolean default False
reboot_behavior String(50) default 'prompt' # prompt | auto | never
maintenance_window_start String(10) # HH:MM
maintenance_window_end String(10)
excluded_software JSON # list of software name patterns to skip
is_active Boolean default True

# PatchRecord
id Integer PK
device_id FK → devices
policy_id FK → patch_policies (nullable)
patch_name String(500)
kb_id String(50)
patch_type String(50) # critical | security | definition | update | feature | driver
status String(50) default 'pending' # pending | approved | deployed | failed
deployed_at DateTime (nullable)
created_at DateTime
```

AutomationProfile + ScheduledTaskRun

```
# AutomationProfile
id Integer PK
name String(255) NOT NULL
customer_id FK → customers (nullable)
schedule_type String(50) # daily | weekly | monthly | once
schedule_day String(20) # monday..sunday or day-of-month
schedule_time String(10) # HH:MM
os_patch_config JSON # {install_all, critical, security, definition, ...}
software_patch_config JSON # {update_all, excluded: [...]}
disk_config JSON # {defrag: bool, checkdisk: bool}
maintenance_config JSON # {delete_temp, restore_point, clear_history, reboot, shutdown}
notification_emails String(500)
run_on_new_agents Boolean default False
is_active Boolean default True
last_run_at DateTime (nullable)
created_at DateTime

# ScheduledTaskRun
id Integer PK
profile_id FK → automation_profiles
device_id FK → devices
status String(20) default 'pending'
result_summary JSON
started_at DateTime (nullable)
finished_at DateTime (nullable)
created_at DateTime
```

Report

```
id Integer PK
name String(255)
template_type String(100) # device_health | patch_compliance | alert_summary |
software_inventory | ticket_summary
customer_id FK → customers (nullable)
format String(10) default 'csv'
file_path String(500) # absolute path to generated CSV in api/reports/
generated_by FK → users (nullable)
created_at DateTime

# IMPORTANT: to_dict() must include "file_path" for dashboard download to work
```

Invoice (invoices)

```
id String/UUID PK
customer_id FK → customers
invoice_number String(20) UNIQUE nullable index # INV-YYYY-NNNN, auto-generated on create
period_start DateTime(timezone=True) NOT NULL
period_end DateTime(timezone=True) NOT NULL
due_date DateTime(timezone=True) nullable # defaults to period_end + 30 days
device_count Integer
per_device_rate Numeric(10,2) NOT NULL
tax_rate Numeric(5,4) default 0.0 # stored as decimal: 0.20 = 20%
notes Text nullable # per-invoice custom notes
subtotal Numeric(10,2) # device_count × per_device_rate
tax_amount Numeric(10,2) # subtotal × tax_rate
total Numeric(10,2) # subtotal + tax_amount
status String(20) default 'draft' # draft | sent | paid | overdue
sent_at DateTime nullable
paid_at DateTime nullable
created_at DateTime
```

Sequential invoice number generation (api/routes/billing.py:_next_invoice_number()):

```
year = datetime.now(timezone.utc).year
count = Invoice.query.filter(Invoice.invoice_number.like(f"INV-{year}-%")).count()
return f"INV-{year}-{count + 1:04d}"
```

OrgSettings (org_settings)

Singleton model — always id=1. Seeded at API startup by ensure_org_settings().

```
id Integer PK default 1 company_name String(255) default "" company_address Text default ""
company_email String(255) default "" company_phone String(50) default "" logo_data Text nullable #
base64 data URI (same pattern as user avatar) payment_terms String(100) default "Net 30" bank_details
Text default "" footer_notes Text default "Thank you for your business!"
```

White-label branding

app_name String(100) default "RMM System" tagline String(200) default "Remote Monitoring & Management" primary_color String(7) default "#407E3C"

Regional settings (added migration f1a2b3c4d5e6, down_revision e6f7a8b9c0d1)

currency String(3) nullable server_default='USD' # ISO 4217 code timezone String(50) nullable server_default='UTC' # IANA tz name

``logo_bytes()`` method decodes the base64 data URI and returns raw bytes for use in PDF generation.

``ensure_org_settings()`` is called from ``create_app()`` – safe to call multiple times (no-op if row exists).

``to_dict()`` returns ``currency`` (default ``"USD"``) and ``timezone`` (default ``"UTC"``) alongside all other fields. Dashboard reads these from ``st.session_state["_org_settings"]`` (loaded once per session in ``require_auth()``).

3. API Endpoints

All endpoints prefixed with ``/api/``. Protected by ``@jwt_required()`` unless noted.

Auth (``/api/auth/``)

Method	Path	Auth	Description
POST	<code>`/login`</code>	None	Email+password → access+refresh tokens, OR <code>`{"status": "mfa_required", "mfa_token": "...}"`</code> (HTTP 200) if user has <code>`mfa_enabled=True`</code>
POST	<code>`/refresh`</code>	Refresh token	Rotate access token
POST	<code>`/logout`</code>	JWT	Revoke token
POST	<code>`/me/force-change-password`</code>	JWT	Set new password + clear <code>`must_change_password`</code> flag
PUT	<code>`/me/password`</code>	JWT	Change password (current + new). Used by Profile page.
POST	<code>`/mfa/setup`</code>	JWT	Generate provisional TOTP secret. Returns <code>`{secret, provisioning_uri}`</code> . Not active until <code>`/mfa/enable`</code> called.
POST	<code>`/mfa/enable`</code>	JWT	Activate MFA. Body: <code>`{code}`</code> . Requires valid TOTP code.
POST	<code>`/mfa/login`</code>	None	Second login step. Body: <code>`{mfa_token, code}`</code> . Returns full JWT. Rate-limited 10/min.
POST	<code>`/mfa/disable`</code>	JWT	Disable MFA. Body: <code>`{password}`</code> . Requires current password.

Agents (``/api/agents/``)

Method	Path	Auth	Description
POST	<code>`/register`</code>	None (org_token)	Register new agent, get device_id + agent_token. Rate-limited: 10/min.
POST	<code>`/<device_id>/heartbeat`</code>	X-Agent-Token	Submit metrics, get queued tasks back
GET	<code>`/<device_id>/tasks`</code>	X-Agent-Token	Poll pending ScriptRun records

```

| POST | `/<device_id>/task_result` | X-Agent-Token | Submit ScriptRun result |
| PUT | `/<device_id>/patches` | X-Agent-Token | Submit pending Windows patches list |
| PUT | `/<device_id>/software` | X-Agent-Token | Submit installed software list |

### Devices (`/api/devices/`)
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/` | JWT | List devices (paginated, filterable) |
| GET | `/<id>` | JWT | Get device + latest metrics |
| PUT | `/<id>` | JWT admin/tech | Update display_name, group_id |
| DELETE | `/<id>` | JWT admin | Remove device |
| GET | `/<id>/metrics` | JWT | Historical metrics (up to 168h). Row cap: `.limit(5000)` prevents unbounded response. |
| GET | `/<id>/software` | JWT | Installed software list |
| POST | `/<id>/reboot` | JWT admin/tech | Queue reboot task |
| POST | `/<id>/shutdown` | JWT admin/tech | Queue shutdown task |
| POST | `/<id>/queue_task` | JWT admin/tech | Queue any built-in maintenance task |
| POST | `/<id>/deploy_patches` | JWT admin/tech | Trigger patch deployment via Celery |
| GET | `/platform_counts` | JWT | Returns `{"by_platform": {windows: N, ...}, "agentless": N}` |
| POST | `/<id>/ping_check` | JWT admin/tech | Ping device IP, update is_online/last_seen. Only for is_agentless=True devices. |

**`queue_task` valid `task_type` values:**
`clean_temp`, `defrag`, `check_disk`, `restore_point`, `clear_browser`, `reboot`, `shutdown`

### Alerts (`/api/alerts/`)
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/` | JWT | List alerts (filter by status, severity) |
| GET | `/rules` | JWT | List alert rules |
| POST | `/rules` | JWT admin/tech | Create alert rule |
| PUT | `/rules/<id>` | JWT admin/tech | Update alert rule |
| DELETE | `/rules/<id>` | JWT admin | Delete alert rule |
| POST | `/<id>/acknowledge` | JWT | Acknowledge alert |
| POST | `/<id>/resolve` | JWT | Resolve alert |

### Tickets (`/api/tickets/`)
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/` | JWT | List tickets (filter by status, priority) |
| POST | `/` | JWT tech/admin | Create ticket (was unguarded – any JWT; now requires `technician` or `admin` role) |
| GET | `/<id>` | JWT | Get ticket + comments |
| PUT | `/<id>` | JWT tech/admin | Update status, priority, assignee_id |
| DELETE | `/<id>` | JWT admin | Delete ticket |
| POST | `/<id>/comments` | JWT | Add comment |

### Scripts (`/api/scripts/`)
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/` | JWT | List scripts (filter built-in/custom) |

```

```
| POST | `/` | JWT tech/admin | Create script |
| PUT | `/<id>` | JWT tech/admin | Update script |
| DELETE | `/<id>` | JWT admin | Delete script |
| POST | `/<id>/run` | JWT tech/admin | Create ScriptRun records for target devices |
| GET | `/runs` | JWT | List recent ScriptRun records |
| GET | `/runs/<id>` | JWT | Get ScriptRun detail (stdout, stderr) |
```

Patches (`/api/patches/`)

```
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/` | JWT | List patch records (filter by status, device) |
| PUT | `/<id>/approve` | JWT tech/admin | Approve a patch record |
| POST | `/approve_bulk` | JWT tech/admin | Approve multiple patch records |
| GET | `/policies` | JWT | List patch policies |
| POST | `/policies` | JWT admin | Create patch policy |
| PUT | `/policies/<id>` | JWT admin | Update patch policy |
```

Automation (`/api/automation/`)

```
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/profiles` | JWT | List automation profiles |
| POST | `/profiles` | JWT admin/tech | Create profile |
| PUT | `/profiles/<id>` | JWT admin/tech | Update profile |
| DELETE | `/profiles/<id>` | JWT admin | Delete profile |
| POST | `/profiles/<id>/run` | JWT admin/tech | Trigger immediate profile run |
| GET | `/runs` | JWT | List ScheduledTaskRun records |
```

Reports (`/api/reports/`)

```
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/` | JWT | List reports |
| POST | `/` | JWT admin/tech | Create report + trigger generation |
| GET | `/<id>` | JWT | Get report metadata (includes file_path) |
```

Billing (`/api/billing/`)

```
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/invoices` | JWT | List invoices (optional `?customer_id=`) |
| POST | `/invoices` | JWT admin | Generate invoice – auto-assigns `invoice_number` (INV-YYYY-NNNN), accepts `tax_rate`, `due_date`, `notes` |
| GET | `/invoices/<id>` | JWT | Get single invoice |
| PUT | `/invoices/<id>/status` | JWT admin | Update invoice status (draft→sent→paid/overdue) |
| DELETE | `/invoices/<id>` | JWT admin | Delete invoice |
| GET | `/invoices/<id>/pdf` | JWT | Generate and stream A4 PDF (Content-Disposition: attachment) |
| POST | `/invoices/<id>/send-email` | JWT admin | Email PDF to customer via SMTP. Returns 503 if `SMTP_HOST` not configured. |
```

Org Settings (`/api/admin/`)

```
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/org-settings` | JWT admin | Return OrgSettings singleton (includes `logo_data`
```

```

base64 URI, `currency`, `timezone`) |
| PUT | `/org-settings` | JWT admin | Update text fields: company name, address, email,
phone, payment terms, bank details, footer notes, `app_name`, `tagline`, `primary_color`,
**`currency`** (ISO 3-letter), **`timezone`** (IANA name) |
| PUT | `/org-settings/logo` | JWT admin | Upload logo – multipart file upload,
Pillow-resized to max 400px wide, stored as base64 |
| DELETE | `/org-settings/logo` | JWT admin | Remove logo (sets `logo_data` = null) |

### Customers (`/api/customers/`)
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/` | JWT | List customers |
| POST | `/` | JWT admin/tech | Create customer |
| GET | `/{<id>}` | JWT | Get customer detail |
| PUT | `/{<id>}` | JWT admin/tech | Update customer |
| DELETE | `/{<id>}` | JWT admin | Deactivate customer |

### Users (`/api/users/`)
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/` | JWT admin | List users |
| POST | `/` | JWT admin | Create user |
| PUT | `/{<id>}` | JWT admin | Update user (role, active, password) |

### Admin (`/api/admin/`)
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/users` | JWT admin | List all users |
| POST | `/users` | JWT admin | Create user – accepts `must_change_password` bool |
| PUT | `/users/<id>` | JWT admin | Update user – accepts `must_change_password` bool.
Returns 403 if target is superadmin. |
| DELETE | `/users/<id>` | JWT admin | Delete user. Returns 403 if target is superadmin.
|
| GET | `/org-token` | JWT admin | Return `ORG_REGISTRATION_TOKEN` for display in Admin
panel |
| GET | `/server_ips` | None | Returns `{"hostname": "...", "lan_ips": ["192.168.x.x"]}`
via socket.getaddrinfo |

### Network (`/api/network/`)
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| POST | `/scan` | JWT admin/tech | Trigger network discovery scan (dispatches
`run_network_scan.delay`) |
| GET | `/scans` | JWT | List scan results |
| POST | `/agentless_devices` | JWT admin/tech | Batch upsert agentless devices. Body:
`{"hosts": [...], "customer_id": optional}`. Returns `{"created": N, "updated": M,
"skipped": K}`. |

### Dashboard Summary (`/api/dashboard/`)
| Method | Path | Auth | Description |
|-----|-----|-----|-----|
| GET | `/summary` | JWT | Aggregated counts (devices, alerts, tickets, online) |
| GET | `/health_map` | JWT | Device health list (limit 500) – returned objects include
`id`, `hostname`, `status`, `is_online`; used by Dashboard Overview to render clickable

```

```
cards |
| GET | `/activity` | JWT | Recent audit log entries |

---

## 4. Authentication & Authorization

### JWT Flow
```

Login (no MFA): POST /api/auth/login {email, password} → {access_token, refresh_token, user: {id, email, role, full_name, must_change_password}}

Login (MFA enabled): POST /api/auth/login {email, password} → HTTP 200 {status: "mfa_required", mfa_token: "<short-lived JWT>"} mfa_token is a 5-min TTL JWT with claim {purpose: "mfa_pending"}. Cannot be used for any other API call. Client must immediately call: POST /api/auth/mfa/login {mfa_token, code} → {access_token, refresh_token, user: {...}}

Force password change (if must_change_password=True): Dashboard intercepts in app.py before showing any page. User fills new password form → POST /api/auth/me/force-change-password {new_password} → clears must_change_password flag → st.rerun() → normal dashboard

Dashboard stores: st.session_state["access_token"] = access_token st.session_state["refresh_token"] = refresh_token st.session_state["user"] = user dict st.query_params["tok"] = access_token ← written once at login st.query_params["rtok"] = refresh_token ← written once at login (Tokens are stripped from URL by require_auth() after first restore. Session state carries them from that point. Mitigates browser history exposure.)

API call: Authorization: Bearer <access_token>

On 401: POST /api/auth/refresh (Authorization: Bearer <refresh_token>) → {access_token} Update session_state, retry original request once

On page reload (F5) or navigation: Streamlit session state is wiped on browser reload. app.py reads st.query_params["tok"] and ["rtok"] before session state check. require_auth() in every page also re-reads and re-stamps both params. require_auth() re-stamps ?tok= and ?rtok= on every authenticated page load — ensures F5 on any page restores both access and refresh tokens, allowing silent renewal even after the 900s access token TTL expires.

```
> **NOTE:** Both access (`?tok=`) and refresh (`?rtok=`) tokens are stored in URL params.
> This is a pragmatic trade-off for Streamlit's stateless page reload model – without it,
> any F5 reload logs the user out. Mitigated by short access token TTL (900s default).
> In production with HTTPS, tokens in URL are not visible in transit but appear in server
> access logs – rotate `JWT_ACCESS_TOKEN_EXPIRES` to 300s for higher-security
environments.

### MFA Flow
```

Setup: POST /api/auth/mfa/setup → {secret, provisioning_uri} (provisional -- not active yet)

Enable (after QR scan): POST /api/auth/mfa/enable {code: "123456"} → {message: "MFA enabled"} -- sets mfa_enabled=True on User

Login with MFA: POST /api/auth/login {email, password} → {status: "mfa_required", mfa_token: "<5-min JWT>"} (HTTP 200) mfa_token JWT claims: {purpose: "mfa_pending", sub: user_id} Cannot be used for any endpoint except /mfa/login.

POST /api/auth/mfa/login {mfa_token, code} Rate-limited: 10/min → {access_token, refresh_token, user: {...}} (full session)

Disable: POST /api/auth/mfa/disable {password} → {message: "MFA disabled"} -- requires current password for confirmation

```
**Dashboard flow:**
- `login()` in `dashboard/utils/auth.py` returns `ok` / `mfa_required` / `error` (not bool).
- On `mfa_required`: stores `mfa_token` in `st.session_state["mfa_pending_token"]`.
- `app.py` route block checks `mfa_pending_token` before `access_token` – shows `show_mfa_step()` full-screen form.
- Back button clears `mfa_pending_token` and returns to login.

**QR code generation:** `qrcode[pil]==8.0` in `dashboard/requirements.txt`.

---

### Agent Token Flow
```

Agent registers: POST /api/agents/register Body: {hardware_id, hostname, platform, os_name, cpu_brand, cpu_cores, ram_gb, ip_local, mac_address, org_token} → {device_id, agent_token}

Stored in: agent/agent_state.json

Subsequent requests: Header: X-Agent-Token: <agent_token>

Token stored hashed in agent_tokens table. On 401: agent re-registers (full registration flow).

```
### Role Permissions Matrix

| Action | viewer | technician | admin | superadmin |
|-----|-----|-----|-----|-----|
| View dashboard, devices, alerts | ✓ | ✓ | ✓ | ✓ |
| Create/update tickets | – | ✓ | ✓ | ✓ |
| Acknowledge/resolve alerts | – | ✓ | ✓ | ✓ |
| Run scripts | – | ✓ | ✓ | ✓ |
| Queue maintenance tasks | – | ✓ | ✓ | ✓ |
| Approve/deploy patches | – | ✓ | ✓ | ✓ |
| Manage automation profiles | – | ✓ | ✓ | ✓ |
| Create/update customers | – | ✓ | ✓ | ✓ |
| Generate reports | – | ✓ | ✓ | ✓ |
| Create/manage users | – | – | ✓ | ✓ |
| Delete any resource | – | – | ✓ | ✓ |
| View Admin page | – | – | ✓ | ✓ |
| Manage billing/invoices | – | – | ✓ | ✓ |
| Modify/delete superadmin account | – | – | – | – (CLI only) |
```

```
> **NOTE:** All `_require_role()` guards in all 9 route files include a superadmin
bypass: `if claims.get("role") == "superadmin": return None`. The superadmin account
itself cannot be modified or deleted via any API endpoint – use `api/reset_superadmin.py`
for password changes.
```

```
---
```

5. Celery Tasks & Schedules

Beat Schedule

Task	Schedule	Module
`evaluate_all_rules`	Every 60s	`tasks.alert_tasks`
`mark_offline_devices`	Every 180s	`tasks.alert_tasks`
`sync_patch_status`	Every 1800s	`tasks.patch_tasks`
`ping_agentless_devices`	Every 300s	`tasks.network_tasks`
`backup_database`	Every 86400s	`tasks.backup_tasks`
`generate_recurring_invoices`	Every 86400s	`tasks.billing_tasks`

Task Signatures

```
##### `evaluate_all_rules()`
- Fetches all active alert rules
- **N+1 fix:** collects all device IDs per rule batch, then loads all latest metrics
in a single subquery:
```

```
max_id_subq = db.select(func.max(DeviceMetrics.id).label("max_id"))\
.where(...).group_by(DeviceMetrics.device_id).subquery() latest_by_device = {m.device_id: m for m in
DeviceMetrics.query.join(max_id_subq, ...).all()}
```

Previously fired one DB query per device per rule (up to 5,000 queries/cycle).
Now: 1 query per rule batch regardless of device count.

- Evaluates `metric operator threshold` (e.g., `cpu\_pct > 90`)
- Respects `cooldown\_minutes` using two separate maps: `open\_alert\_map` (no time filter, prevents re-fire while alert is open) and `resolved\_alert\_map` (recently resolved, for post-recovery cooldown). Fixes storm where alerts older than cooldown_minutes were evicted and re-fired every 60s indefinitely.
- ****Auto-resolve:**** when metric drops back below threshold, any open alerts for that rule+device are resolved automatically (`status="resolved", `resolved_at=now()`).
- Creates `Alert` record on breach
- Alert message includes last-seen timestamp for offline alerts:
`"DESKTOP-ABC has gone offline (last seen: 2024-01-15 14:32 UTC)"`
- Calls `send\_alert\_notification()` if `notification\_channels.email` is set

```
##### `mark_offline_devices()`
- Marks devices as `is_online=False` if `last_seen` > 5 minutes ago
- Creates offline alerts if a matching rule exists
```

```
##### `sync_patch_status()`
- Iterates all active `PatchPolicy` records
- For each policy's scope (customer or global):
- Auto-approves `PatchRecord` rows where `status='pending'` matching policy flags
- Respects `excluded_software` name patterns
```

```

#### `deploy_patches(device_id, patch_ids)`
- Looks up `PatchRecord` rows by ID
- Builds a PowerShell script using PSWindowsUpdate to install those KB IDs
- Creates or updates a `Script` record with tag `__deploy_patches_transient__`
- Creates a `ScriptRun` pointing to that script for the device
- Marks `PatchRecord.status = 'deployed'`

#### `generate_report(report_id)`
- Loads `Report` record by ID
- Dispatches to template-specific data collector:
- `device_health` – all devices + batch-fetched latest metrics
- `patch_compliance` – all patch records + device/policy joins
- `alert_summary` – all alerts + rule names + device hostnames
- `software_inventory` – all installed software + device hostnames
- `ticket_summary` – all tickets + customer names
- Writes CSV to `api/reports/<report_id>_<name>.csv`
- Updates `report.file_path` in DB

#### `enqueue_profile_run(profile_id)`
- Loads `AutomationProfile`
- Queries all devices in scope (by `customer_id` or global)
- Calls `_dispatch_profile_tasks(profile, device_id, db_session)` per device
- Updates `profile.last_run_at`

#### `_dispatch_profile_tasks(profile, device_id, db_session)`
- Reads `disk_config` and `maintenance_config` JSON
- For each enabled task type, creates a `ScriptRun` pointing to the corresponding built-in script
- Also creates `ScriptRun` records for any custom scripts attached to the profile

#### `run_network_scan(scan_id)`
- Parses CIDR range from the `NetworkScan` record
- Concurrent ICMP ping sweep via `ThreadPoolExecutor(50)` – all IPs in range pinged in parallel
- For each live IP: ARP table lookup for MAC address, OUI vendor lookup via `lookup_vendor(mac)`
- Reverse DNS via `_get_hostname(ip)` runs before port probe so hostname is available for all fallbacks
- Detection pipeline (stops at first match):
1. `_guess_platform(vendor)` – OUI vendor string keywords
2. `_probe_platform(ip)` – TCP ports: 62078→iOS, 5555→Android, 2+ of** (445/3389/139)→Windows (2-port threshold prevents routers/NAS with only SMB from being misidentified), 548→macOS, 22→Linux; skipped entirely for router hostnames
3. `_guess_platform_from_hostname(rdns)` – 50+ keyword match on rDNS hostname (brand names + Samsung model numbers S10-S24, Note, A12-A73, Fold, Flip, Ultra); catches Android phones with ADB disabled
- Router/gateway hostnames (`fritz.box`, `router`, `gateway`, `modem`, etc.) are included in `discovered_hosts` for scan display but never persisted as agentless device records
- Calls `_upsert_agentless_host(...)` for all non-router live hosts
- Updates `NetworkScan` record with status, host count, and completion timestamp

```

```
`_upsert_agentless_host` lookup order (stops at first match):
1. MAC address – primary stable key
2. IP address across ALL devices (not just agentless) – prevents duplicates for
agent-managed PCs
3. Bare hostname match (strips `.fritz.box`/`.local`, case-insensitive) – catches
multi-adapter machines where agent and scan see different MACs (e.g. Ethernet vs WiFi
adapter)
- On match with non-agentless device: returns `"skipped"` – never modifies agent-managed
records
- On match with agentless device: upgrades `platform`/`device_type` if currently
"unknown" and new scan detects better info
```

```
> **NOTE:** Phones with fully randomized MACs and all ports blocked will still reach the
hostname fallback. If the router assigns a model-name hostname (Fritz!Box, ASUS, TP-Link
all do), the device will be identified. Devices that remain "unknown" after all three
stages can be corrected via the Edit button on the agentless device row.
```

```
#### Stale Network Scan Cleanup (startup, not Celery)
```

```
Network scans run in a `threading.Thread(daemon=True)` – if Flask restarts mid-scan, the
thread is killed but the `NetworkScan` DB record stays in `"running"` forever. On every
`create_app()` call, a cleanup block runs **before** the API accepts requests:
```

```
cutoff = datetime.now(timezone.utc) - timedelta(minutes=5) stale = NetworkScan.query.filter(
NetworkScan.status == "running", NetworkScan.started_at < cutoff, ).all() for s in stale: s.status = "failed"
s.completed_at = datetime.now(timezone.utc) s.discovered_hosts = [{"error": "Scan interrupted (server
restarted)"}] db.session.commit()
```

```
This resolves all orphaned scans automatically on every API start. No manual DB
intervention needed.
```

```
#### `ping_agentless_devices()`
- Loads all `Device` records where `is_agentless=True`
- Pings each device's `ip_address`
- Updates `is_online=True` and `last_seen=now()` on response
- Devices that have not responded for more than 10 minutes are marked `is_online=False`
- Changes committed in a single batch after all pings complete
```

```
### Starting Celery (Windows)
```

Worker (must use --pool=solo on Windows — no fork())

```
celery -A tasks.celery_app worker --pool=solo -l info
```

Beat scheduler (separate process)

celery -A tasks.celery_app beat -l info

```
---

## 6. Agent Internals

### Cross-Platform Support

`collector.py` dispatches on `sys.platform` for all hardware/software/patch collection:

| Platform | `sys.platform` | Hardware info | Patches | Software |
|-----|-----|-----|-----|-----|
| Windows | `win32` | WMI + winreg | WUA COM via PowerShell | Registry + winget |
| macOS | `darwin` | sysctl + system_profiler + platform.mac_ver() | softwareupdate -l + brew outdated | /Applications plist + brew list |
| Linux | `linux` | /proc/cpuinfo + dmidecode + /etc/os-release | apt-get/dnf/yum/pacman (auto-detected) | dpkg-query/rpm -qa/pacman -Q |

The `platform` field in `hardware_info` is auto-detected (`windows`/`mac`/`linux`) instead of hardcoded. All Windows-specific code (winreg, WMI, DPAPI, winget) is guarded by `_PLATFORM == "win32"` checks.

### PyInstaller Binary Builder

`agent/build.py` produces a single-file executable with no Python dependency on the target machine:
```

pip install pyinstaller python build.py

Output: dist/rmm_agent.exe (Windows) or dist/rmm_agent (Linux/macOS)

```
- `--onefile` mode, auto-detects `--noconsole` on Windows
- Hidden imports per platform: `wmi`/`win32crypt`/`plistlib` included only when relevant
- `config.ini` template copied next to binary after build
- Deploy: copy `dist/` to target, edit `config.ini`, run binary

### Main Loop
```

startup ■■ load agent_state.json (device_id, token) ■■ if not registered: POST /api/agents/register → save state ■■ prime CPU counter: psutil.cpu_percent(interval=1) ■■ loop every 60s: ■■ collect_metrics() (non-blocking cpu_percent(interval=None)) ■■ send_heartbeat(metrics) ■■ on 401: re-register ■■ on connection error: exponential backoff 15s→300s ■■ get_tasks() ■■ executor.run(task) for each queued task ■■ if time – last_patch > patch_interval: ■■ get_pending_patches() (platform-specific) ■■ report_patches(patches) ■■ if time – last_screenshot > screenshot_interval (default 300s): ■■ screenshot.capture() (PIL/scrot, None on headless) ■■ heartbeat.send_screenshot(jpeg_bytes) ■■ flush pending_results.json (local queue)

Task Execution

Agent receives tasks in format:

```
{ "type": "run_script", "run_id": "<uuid>", "script_type": "ps1", "script_content": "...", "timeout": 300 }
```

```
`executor.py` passes to `script_runner.py`:  
- **ps1**: `powershell -NoProfile -NonInteractive -Command <content>` with  
  `CREATE_NO_WINDOW`  
- **bat**: `cmd /c <tempfile.bat>`  
- **py**: `python <tempfile.py>`  
- **sh**: `bash <tempfile.sh>` (Linux/macOS agents)
```

After execution, agent POSTs result to `/api/agents/<id>/task\_result`:

```
{ "run_id": "<uuid>", "exit_code": 0, "stdout": "...", "stderr": "", "status": "completed" }
```

Patch Scanning

Uses PowerShell WUA COM object – requires no external package:

```
$session = New-Object -ComObject Microsoft.Update.Session $searcher =  
$session.CreateUpdateSearcher() $result = $searcher.Search("IsInstalled=0 and Type='Software'")
```

Returns up to 500 patches as `[name, kb\_id, patch\_type]`. Reported via `PUT /api/agents/<id>/patches`. API deduplicates by `patch\_name` and creates `PatchRecord` rows with `status='pending'`.

Software Collection

`get\_installed\_software()` merges two sources, deduplicated by lowercase name:

Source	Function	Notes
Windows registry	`_get_registry_software()`	3 hive paths, 20s hard deadline
winget	`_get_winget_software()`	Skips non-ASCII lines (progress bars); finds separator line before parsing

Winget parser detail: `winget list` prints Unicode block-char progress bars (■■■■, `ord > 127`) before the real data table. The parser skips these lines, finds the first all-dash separator line (length > 10), then parses only lines after it. The dashboard adds a `\_clean()` filter (strips U+2500–U+259F) as defence-in-depth before HTML rendering.

Resilience Features

Feature	Implementation
Non-blocking CPU	`cpu_percent(interval=None)` after startup prime
Bounded process scan	Cap at 200 processes, 3s timeout
Bounded registry scan	20s hard deadline
Winget Unicode parser	Skip `ord > 127` lines; locate separator line before data

```

| Heartbeat backoff | 15s → 30s → 60s ... → 300s cap on failure |
| 401 re-registration | Main loop checks status code, triggers full re-register |
| Local result queue | `pending_results.json`, cap 100, flushed each cycle |
| Structured logging | `DeviceFilter` injects device_id into all log records |

---

## 7. Built-in Scripts Reference

Built-in scripts are `Script` records with `is_builtin=True`. The `description` field
stores the lookup tag. Created/updated at API startup via `ensure_builtin_scripts()`.

| task_type | Tag | Script name | What it does |
|-----|-----|-----|-----|
| `clean_temp` | `__builtin_clean_temp__` | Built-in: Clean Temp Files | Removes
`%TEMP%`, `%TMP%`, `C:\Windows\Temp` |
| `defrag` | `__builtin_defrag__` | Built-in: Defragment | `Optimize-Volume -DriveLetter
C -Defrag` |
| `check_disk` | `__builtin_check_disk__` | Built-in: Check Disk | `chkdsk C: /f` –
schedules for next reboot if drive locked; exits 0 in all cases |
| `restore_point` | `__builtin_restore_point__` | Built-in: Create Restore Point |
`Checkpoint-Computer -Description "RMM"` |
| `clear_browser` | `__builtin_clear_browser__` | Built-in: Clear Browser History |
Removes Chrome/Edge/Firefox cache dirs |
| `reboot` | `__builtin_reboot__` | Built-in: Reboot | `Restart-Computer -Force` |
| `shutdown` | `__builtin_shutdown__` | Built-in: Shutdown | `Stop-Computer -Force` |

**Adding a new built-in script:**
1. Add entry to `TASK_TYPE_TO_TAG` dict in `api/utils/builtin_scripts.py`
2. Add entry to `BUILTIN_SCRIPTS` dict with `name`, `content`, `file_type`
3. Restart the API – `ensure_builtin_scripts()` will create the DB record
4. Add the new `task_type` to any dispatch logic that should use it

**Transient scripts** (not in the built-in registry):
- `__deploy_patches_transient__` – created dynamically by `deploy_patches` Celery task
per deployment run

---

## 8. Dashboard API Client

`dashboard/utils/api_client.py` – `RMMClient` class

### Session Management

```

One RMMClient per browser session, stored in st.session_state

```
def get_client(): if "_rmm_client" not in st.session_state: st.session_state["_rmm_client"] = RMMClient(...)
return st.session_state["_rmm_client"]
```

```
### Retry + 401 Refresh
```

```
def _request(self, method, path, **kwargs): for attempt in range(3): try: resp =
self.session.request(method, url, ...) if resp.status_code == 401 and attempt == 0: self._refresh_token()
continue return resp.json(), None except (ConnectionError, Timeout): time.sleep([0.5, 1.0, 2.0][attempt])
return None, "Service unavailable"
```

```
### Key Methods
```

```
| Method | HTTP | Path |
|-----|-----|-----|
| `list_devices(page, per_page, **filters)` | GET | `/api/devices/` |
| `get_device(device_id)` | GET | `/api/devices/<id>` |
| `queue_device_task(device_id, task_type, timeout=300)` | POST |
`/api/devices/<id>/queue_task` |
| `reboot_device(device_id)` | POST | `/api/devices/<id>/reboot` |
| `shutdown_device(device_id)` | POST | `/api/devices/<id>/shutdown` |
| `deploy_patches(device_id, patch_ids)` | POST | `/api/devices/<id>/deploy_patches` |
| `list_tickets(**filters)` | GET | `/api/tickets/` |
| `create_ticket(data)` | POST | `/api/tickets/` |
| `update_ticket(ticket_id, data)` | PUT | `/api/tickets/<id>` |
| `add_comment(ticket_id, body, is_internal)` | POST | `/api/tickets/<id>/comments` |
| `list_alerts(**filters)` | GET | `/api/alerts/` |
| `list_customers(per_page)` | GET | `/api/customers/` |
| `list_users()` | GET | `/api/users/` |
| `force_change_password(new_password)` | POST | `/api/auth/me/force-change-password` |
| `change_password(current_password, new_password)` | PUT | `/api/auth/me/password` |
| `mfa_setup()` | POST | `/api/auth/mfa/setup` |
| `mfa_enable(code)` | POST | `/api/auth/mfa/enable` |
| `mfa_disable(password)` | POST | `/api/auth/mfa/disable` |
| `mfa_login(mfa_token, code)` (static) | POST | `/api/auth/mfa/login` |
| `list_patches(**filters)` | GET | `/api/patches/` |
| `approve_patches(patch_ids)` | POST | `/api/patches/approve_bulk` |
| `list_profiles()` | GET | `/api/automation/profiles` |
| `run_profile(profile_id)` | POST | `/api/automation/profiles/<id>/run` |
| `create_report(data)` | POST | `/api/reports/` |
| `list_reports()` | GET | `/api/reports/` |
| `get_platform_counts()` | GET | `/api/devices/platform_counts` |
| `ping_check_device(device_id)` | POST | `/api/devices/<id>/ping_check` |
| `upsert_agentless_devices(hosts, customer_id=None)` | POST |
`/api/network/agentless_devices` |
| `trigger_network_scan(customer_id, scan_range)` | POST | `/api/network/scan` |
| `get_server_ips()` | GET | `/api/admin/server_ips` |
| `update_device(device_id, data)` | PUT | `/api/devices/<id>` |
| `delete_device(device_id)` | DELETE | `/api/devices/<id>` |
| `list_invoices(customer_id=None)` | GET | `/api/billing/invoices` |
```

```
| `get_invoice(invoice_id)` | GET | `/api/billing/invoices/<id>` |
| `generate_invoice(data)` | POST | `/api/billing/invoices` |
| `update_invoice_status(invoice_id, status)` | PUT | `/api/billing/invoices/<id>/status`
|
| `delete_invoice(invoice_id)` | DELETE | `/api/billing/invoices/<id>` |
| `get_invoice_pdf_bytes(invoice_id)` | GET | `/api/billing/invoices/<id>/pdf` – returns
`(bytes, error)`, bypasses JSON parsing |
| `send_invoice_email(invoice_id)` | POST | `/api/billing/invoices/<id>/send-email` |
| `get_org_settings()` | GET | `/api/admin/org-settings` |
| `update_org_settings(data)` | PUT | `/api/admin/org-settings` |
| `upload_org_logo(file_bytes, content_type)` | PUT | `/api/admin/org-settings/logo` –
uses `requests.put(files=)`, bypasses JSON headers |
| `delete_org_logo()` | DELETE | `/api/admin/org-settings/logo` |
```

HTTPS Warning

`api\_client.py` logs a `WARNING` at import time if `API\_BASE\_URL` uses `http://` pointing to a non-localhost host. This warns that tokens and data will be transmitted unencrypted.

Cache Strategy

@st.cache_data(ttl=30) # fast-changing: devices, alerts @st.cache_data(ttl=60) # medium: tickets, customers @st.cache_data(ttl=120) # slow-changing: scripts, users, patch policies

9. Performance Optimizations

Database

```
| Optimization | Location | Detail |
|-----|-----|-----|
| Composite indexes | Direct SQL (post-migration) | 7 new indexes:
`ix_device_metrics_device_collected_at`, `ix_devices_customer_online`,
`ix_alerts_device_status`, `ix_alerts_status_triggered_at`,
`ix_patch_records_device_status`, `ix_tickets_customer_status`,
`ix_script_runs_device_status` |
| N+1 alert evaluation fix | `tasks/alert_tasks.py` | `evaluate_all_rules()` now loads
all latest metrics per rule in 1 subquery. Was up to 5,000 queries/cycle (50 rules x 100
devices). |
| Metrics endpoint row cap | `routes/devices.py` | `GET /api/devices/<id>/metrics`
applies `.limit(5000)`. Was unbounded (10,080+ rows for 7-day window). |
| Batch latest metrics | `routes/devices.py:batch_latest_metrics()` | Subquery MAX(id)
per device – 1 query instead of N |
| Dashboard aggregation | `routes/dashboard.py` | 4 DB aggregations replace 8 COUNT
queries |
| Health map cap | `routes/dashboard.py` | `.limit(500)` prevents full-table load |
| N+1 agent tasks | `routes/agents.py` | Pre-fetch scripts by id dict before loop |
| DB connection pool | `config.py` | `pool_size=10, max_overflow=20, pool_pre_ping=True`
|
```

Celery

```
| Optimization | Detail |
|-----|-----|
| `acks_late=True` | Task not acknowledged until completion – survives worker crash |
| `OperationalError` retry | DB connection drops retry with exponential backoff |
| Automation offloaded | Device-loop in `enqueue_profile_run` runs in Celery, not in request thread |
```

API Server

```
| Optimization | Detail |
|-----|-----|
| gzip compression | `flask-compress` gzips all JSON responses >500B. health_map: 5KB→1.2KB, activity_feed: 40KB→877B |
| Waitress threads | 16 threads (up from 8) – headroom for Celery beat contention |
| JWT decode cache | `api/utils/jwt_cache.py` – `TTLCache(512, 60s)` monkey-patches `_decode_jwt_from_request`. Repeated tokens skip HMAC: 60-86ms vs 135-177ms.
Process-local, no Redis needed. |
| Raw JSON cache | `cache_get_raw`/`cache_set_raw` stores pre-serialized JSON bytes in Redis. `health_map` returns `Response(raw, mimetype="application/json")` – bypasses double serialization. |
| Cache pre-warm | `create_app()` pre-populates `rmm:dash:summary:all` at startup so first user hits Redis not DB. Cold hits: ~1300ms → ~100-260ms. |
| PgBouncer | Route `DATABASE_URL` through PgBouncer on port 6432 for persistent backend connections. `pool_pre_ping` disabled (saves ~100ms per cold acquisition; `pool_recycle=300` covers stale connections). |
| PostgreSQL work_mem | `ALTER ROLE rmm_app SET work_mem = '16MB'` – improves sort/hash query performance. |
```

Dashboard

```
| Optimization | Detail |
|-----|-----|
| `RMMClient` session reuse | One `requests.Session` per browser session – persistent HTTP connection |
| Retry backoff | `ConnectionError`/`Timeout` only – 0.5s/1.0s/2.0s |
| `dashboard/utils/cached_calls.py` | `st.cache_data` wrappers for all major read endpoints (`cached_summary` TTL 60s, `cached_health_map` TTL 30s, `cached_list_devices` TTL 30s, etc.). Call `st.cache_data.clear()` after mutations. |
| `st.fragment` auto-refresh | `01_Dashboard.py` body wrapped in `@st.fragment(run_every=30)` – no longer blocks server thread. `14_Terminal.py` output panel wrapped in `@st.fragment(run_every=2)`. |
| Parallel API calls | `04_Devices.py` and `10_Admin.py` use `ThreadPoolExecutor` to fire independent API calls in parallel. |
| `st.spinner` | All data loads wrapped – user sees progress feedback |
| Graceful degradation | `st.warning` instead of `st.stop()` – page stays interactive on partial failure |
| Token URL security | `require_auth()` strips `?tok=``?rtok=` from URL after first restore. Tokens no longer re-stamped on every page load – reduces browser history exposure. |
| Org settings session cache | `require_auth()` loads `GET /api/admin/org-settings` once per session into `st.session_state["_org_settings"]`. All pages read `currency`/`timezone` from this dict – zero repeated API calls. Admin page clears `_org_settings` from session state after Regional Settings save to force a fresh load. |
```

Metrics History – 7-Day Fallback

``dashboard/pages/04_Devices.py`` – ``_render_agent_row()`` metrics history block.

Default window: 24 hours (``GET /api/devices/<id>/metrics?hours=24``). If the API returns an empty list (agent was offline for >24h), the dashboard retries with ``hours=168`` (7 days). On fallback:

- An ``st.info`` banner shows: `*"No data in last 24 h – showing last N readings (oldest ~Xh ago). Agent may be offline."*`
- Chart title changes from ``"24-hour usage history"`` to ``"7-day usage history (agent offline)"``.
- If the 7-day window is also empty, ``st.warning("No metric history available.")`` is shown.

The ``collected_at`` timezone handling: ``pd.to_datetime()`` preserves tz-aware timestamps; the "hours ago" calculation converts to UTC via ``tz_convert("UTC")`` before diffing against ``pd.Timestamp.now(tz="UTC")``.

Responsive Design – CSS Breakpoints

Responsive styles live entirely in ``dashboard/utils/styles.py`` (``_GLOBAL_CSS_RULES``) and are injected as a static CSS file via ``inject_css()``. The login page adds its own mobile overrides through ``_login_css()`` in ``dashboard/app.py``.

****Breakpoints:****

Breakpoint	Selector	Key rules
≤1024px (tablet)	<code>`@media (max-width: 1024px)`</code>	Padding → 1.25rem, metric value → 1.6rem
≤768px (mobile)	<code>`@media (max-width: 768px)`</code>	Columns stack (<code>`flex-wrap: wrap`</code> , <code>`flex: 1 1 100%`</code>), 44px tap targets, 1rem input font, tabs scroll horizontally, DataFrames scroll, sidebar capped at 82vw, login card full-width
≤480px (small phone)	<code>`@media (max-width: 480px)`</code>	Padding → 0.4rem, heading/metric further reduced

****Column stacking – how it works:****

Streamlit renders ``st.columns()`` as ``[data-testid="stHorizontalBlock"]`` with children ``[data-testid="column"]``. On mobile:

```
[data-testid="stHorizontalBlock"] { flex-wrap: wrap !important; } [data-testid="column"] { width: 100% !important; min-width: 100% !important; flex: 1 1 100% !important; }
```

All column layouts – stat cards, device rows, billing columns – stack vertically. Empty spacer columns (e.g. ``_``, ``col``, ``_`` in login) have no content so they take zero visible height.

****Touch targets (WCAG 2.5.5):****

```
.stButton > button { min-height: 44px !important; } .stTextInput input { min-height: 44px !important; font-size: 1rem !important; }
```

```
`font-size: 1rem` on inputs prevents iOS Safari from auto-zooming on focus.
```

```
**Tab overflow:**
```

```
[data-testid="stTabs"] { overflow-x: auto !important; -webkit-overflow-scrolling: touch !important; }  
[data-testid="stTabs"] [role="tablist"] { flex-wrap: nowrap !important; }
```

Tabs scroll horizontally on mobile instead of wrapping or clipping.

```
**Login page specifics (`_login_css()` in `app.py`):**
```

Same column-stack rules are duplicated inside the login CSS block so they apply before ``inject_css()`` is called. Login-specific: ``min-height: 48px`` on inputs/buttons (slightly larger for first-touch UX), ``brand-title`` shrinks to `1.5rem`.

```
**Platform limitation:** Streamlit injects its own `content="width=device-width, initial-scale=1">` – no override needed. However, Streamlit renders in an iframe context where the `window.innerWidth` seen by CSS is the iframe width, not the device width. On current Chrome/Safari (mobile), this equals the viewport width correctly. On some embedded or WebView contexts, breakpoints may not fire. No workaround exists within Streamlit's current architecture.
```

```
**White-label color swap compatibility:** `inject_css()` does a string-replace of  
`#407E3C` → custom color before writing the CSS file. The responsive rules contain no  
color values, so the swap is unaffected.
```

Device Health Map – Clickable Cards

Cards in ``dashboard/pages/01_Dashboard.py`` use ``st.button()`` + ``st.switch_page()`` for reliable Streamlit-native navigation. Plain ``<a href>`` links and ``components.html()`` with ``window.parent.location.href`` are intercepted/sandboxed by Streamlit's SPA router and do not work.

```
**Navigation flow:**
```

1. User clicks a health map card → ``st.session_state["_nav_device"] = str(device_id)`` → ``st.switch_page("pages/04_Devices.py")``
2. ``04_Devices.py`` reads ``st.session_state.pop("_nav_device", None)`` on load, falls back to ``st.query_params.get("device")`` for direct URL access
3. If matched, pre-fills search box with the device hostname and shows an info banner

```
**CSS scoping:** Card button styles are injected inside the `else` branch of the health  
map render and use `section[data-testid="stMain"] button[kind="secondary"]` to target  
only main-area secondary buttons – sidebar buttons are excluded. Refresh button uses  
`type="primary"` to opt out of card styling entirely.
```

```
---
```

10. Extension Guide

Adding a New Dashboard Page

1. Create ``dashboard/pages/NN_PageName.py``
2. Follow the standard page template:

```
import streamlit as st from utils.auth import require_auth from utils.nav import render_sidebar from
utils.styles import inject_css

st.set_page_config(page_title="Page Name — RMM", layout="wide") inject_css()

client = require_auth() render_sidebar()

st.markdown('<h1 style="margin:0">Page Name</h1>', unsafe_allow_html=True)

with st.spinner("Loading..."): data, err = client.some_method() if err: st.warning(f'Could not load data —
{err}') st.stop()
```

3. Add a nav link in `dashboard/utils/nav.py` under the appropriate section

Adding a New API Route

1. Create `api/routes/mymodule.py`:

```
from flask import Blueprint, request, jsonify from flask_jwt_extended import jwt_required, get_jwt from
extensions import db
```

```
mymodule_bp = Blueprint("mymodule", __name__)
```

```
@mymodule_bp.route("/", methods=["GET"]) @jwt_required() def list_items(): ... return jsonify({"items":
[...]}), 200
```

2. Register in `api/app.py`:

```
from routes import mymodule app.register_blueprint(mymodule.mymodule_bp, url_prefix='/api/mymodule')
```

3. Add corresponding method to `dashboard/utils/api\_client.py`

Adding a New Built-in Script

1. Edit `api/utils/builtin\_scripts.py`:

Add to TASK_TYPE_TO_TAG:

```
"my_task": "__builtin_my_task__",
```

Add to BUILTIN_SCRIPTS:

```
"my_task": { "name": "Built-in: My Task", "content": "## PowerShell script here\nWrite-Output 'Done'",
"file_type": "ps1", },
```

2. Restart the API – `ensure\_builtin\_scripts()` creates the DB record automatically
3. Optionally add the `task\_type` to `\_dispatch\_profile\_tasks()` in
`tasks/automation\_tasks.py`
4. Optionally add a UI button in the relevant dashboard page calling
`client.queue\_device\_task(device\_id, "my\_task")`

Adding WiFi/Agentless Devices

****For Windows/Linux/macOS laptops on the same WiFi/LAN:****

1. Find the server's LAN IP: Admin → System Info → Server IP Addresses, or check your router.
2. Copy the `agent/` folder to the target machine (USB, network share, or git clone).
3. On the target machine, install dependencies:

```
cd agent python -m venv venv venv\Scripts\activate # Windows pip install -r requirements.txt
```

4. Run the setup helper – patches `config.ini` automatically:

```
python setup_agent.py 192.168.x.x <org_token>
```

The org token is shown in Admin → System Info → Agent Enrollment Token.

5. Start the agent:

```
python rmm_agent.py
```

6. The device appears in Devices → Windows/Linux/macOS tab within 60 seconds.

****For phones, tablets, and IoT devices (no agent possible):****

1. Go to Network Discovery in the dashboard.
2. Enter the subnet in CIDR notation (e.g. `192.168.1.0/24`) and optionally assign a customer.
3. Click ****Scan Network**** – the scan takes 15–30 seconds via concurrent ICMP ping.
4. Discovered devices appear with IP, MAC, vendor (from OUI lookup), and detected platform.
 - iOS devices are identified by Apple OUI prefixes, or by port 62078 being open (port probe fallback).
 - Android devices are identified by Samsung/Google/OnePlus/etc. OUI prefixes, or by port 5555 being open.
 - When OUI lookup returns unknown, `\_probe\_platform(ip)` tries 6 TCP ports to detect the OS.
 - Reverse DNS (`\_get\_hostname`) populates a friendly hostname (e.g. `Printer.local`) when available.
5. Click ****Save All to Devices**** to persist them to the database.
6. Devices appear in Devices → Android/iOS/Agentless tabs.
7. Celery beat pings them every 5 minutes – online/offline status updates within 10 minutes of disconnect.

****Identifying devices that evade automated detection (randomized MAC + closed ports):****

Modern phones randomize their MAC address per network, defeating OUI lookup. They also typically block all probed ports. These devices will remain platform "unknown" after a scan. Use the ****Edit**** button on the agentless device row in the Devices page to manually set hostname, platform, and device_type. This calls `PUT /api/devices/<id>` via `client.update\_device(device\_id, data)`.

Adding or Changing a Supported Currency or Timezone

All supported currencies and timezones are defined as Python lists in `dashboard/utils/formatters.py`:

```
SUPPORTED_CURRENCIES = ["USD", "EUR", "GBP", "CHF", "CAD", "AUD", "JPY", "SEK", "NOK", "DKK"]
```

```
CURRENCY_SYMBOLS = { "USD": "$", "EUR": "€", "GBP": "£", "CHF": "Fr", "CAD": "CA$", "AUD": "A$", "JPY": "¥", "SEK": "kr", "NOK": "kr", "DKK": "kr", }
```

```
SUPPORTED_TIMEZONES = [ "UTC", "Europe/Berlin", "Europe/London", ... ]
```

To add a new currency: append the ISO code to `SUPPORTED_CURRENCIES` and add its symbol to `CURRENCY_SYMBOLS`. No DB migration needed – currency/timezone values are stored as plain strings.

To add a new timezone: append the IANA name to `SUPPORTED_TIMEZONES`. Verify Python's `zoneinfo` module recognizes it: `from zoneinfo import ZoneInfo; ZoneInfo("Your/Zone")` (raises `ZoneInfoNotFoundError` if invalid).

`fmt_currency(amount, currency)` – formats a numeric amount with the correct symbol:

```
fmt_currency(49.99, "EUR") # → "€49.99" fmt_currency(49.99, "JPY") # → "¥49.99"
```

`fmt_datetime_tz(iso_str, tz_name)` – converts a UTC ISO string to local time for display:

```
fmt_datetime_tz("2026-06-16T10:00:00Z", "Europe/Berlin") # → "2026-06-16 12:00"
```

```
fmt_datetime_tz("2026-06-16T10:00:00Z", "America/New_York") # → "2026-06-16 06:00"
```

Falls back to `fmt_datetime(iso_str)` (UTC, no conversion) if `zoneinfo` raises or the `tz_name` is invalid. This prevents crashes when an unsupported timezone is stored in the DB.

Consuming regional settings in a new dashboard page:

```
_org = st.session_state.get("_org_settings", {}) _currency = _org.get("currency", "USD") _tz = _org.get("timezone", "UTC")
```

Then use:

```
fmt_currency(invoice["total"], _currency) fmt_datetime_tz(invoice["created_at"], _tz)
```

`_org_settings` is loaded once per session by `require_auth()` and is always available by the time any page body runs.

Adding a New Report Template

1. Add to `tasks/report_tasks.py`:

```
def _my_template(report, db_session): rows = db_session.query(MyModel).all() return [{"col1": r.field1, "col2": r.field2} for r in rows]
```

2. Register in the dispatcher dict inside `generate_report()`:

```
COLLECTORS = { ... "my_template": _my_template, }
```

```
3. Add option to the template dropdown in `dashboard/pages/08_Reports.py`
```

```
---
```

```
## 11. Security Model
```

```
### Network Boundaries
```

```
| Surface | Exposure | Notes |
|-----|-----|-----|
| Streamlit `:8501` | LAN/localhost only | No direct user data – proxies to API |
| Flask API `:5000` | LAN/localhost only | All auth enforced via JWT |
| PostgreSQL `:5432` | localhost only | `rmm_app` user has no superuser |
| Redis `:6379` | localhost only | No password in dev (add one for production) |
```

```
### Authentication Layers
```

1. **Dashboard users** – JWT (HS256, `JWT\_SECRET\_KEY`), 900s access / 7d refresh
2. **Agents** – `X-Agent-Token` header, token stored as SHA256 hash in `agent\_tokens` table
3. **API startup** – `ORG\_REGISTRATION\_TOKEN` required for agent registration
4. **Superadmin** – seeded at every API startup via `ensure\_superadmin()` in `api/utils/superadmin.py`; bypasses all `require\_role()` checks; cannot be deleted or modified via API; password reset via `api/reset\_superadmin.py` CLI

```
### Known Risks & Mitigations
```

```
| Risk | Mitigation |
|-----|-----|
| JWT token theft | Short expiry (900s). HTTPS in production. Tokens written to URL once at login, then stripped by `require_auth()` after first restore – no longer re-stamped on every page load. |
| Script injection | Scripts stored as plain text – reviewed before running. Agent now uses `-ExecutionPolicy RemoteSigned` instead of `Bypass` – locally created scripts run, remote scripts must be signed. |
| Error response leakage | 400/422 handlers no longer return `str(e)` to client. Detail logged server-side only; generic message returned to caller. |
| CORS wildcard | `CORS_ORIGINS` env var (comma-separated) replaces `origins=""`. Default: `http://localhost:8501`. Set to dashboard URL(s) in production. |
| Privilege escalation | Role enforced at API level via `get_jwt()` claims check on every mutation endpoint |
| Celery task injection | Tasks only dispatched by authenticated API endpoints or beat schedule |
| DB credentials in env | `.env` in `.gitignore`. Never committed. |
| Report file exposure | `api/reports/` not served via HTTP. Download reads bytes directly in dashboard. |
| SMTP credential exposure | `SMTP_PASS` read from env only. Never logged. |
| Superadmin lockout | Auto-seeded at every API startup – account always exists. Emergency password reset: `cd api ; python reset_superadmin.py <new_password>` |
```

```
### Production Checklist
```

```
- [ ] Set `FLASK_DEBUG=0`
```

- [] Use HTTPS (nginx reverse proxy with Let's Encrypt)
- [] Add Redis password: `requirepass <password>` in Memurai config
- [] Rotate `ORG\_REGISTRATION\_TOKEN` after all agents registered
- [] Change default admin password from `Admin1234!`
- [] Set up automated PostgreSQL backups
- [] Move `api/reports/` to a dedicated volume outside the project root
- [] Run agent as a dedicated low-privilege Windows service account (not SYSTEM)
- [] Enable MFA for all admin accounts (full TOTP implementation: setup, enable, login, disable via Profile page)
- [] Set `CORS\_ORIGINS` to dashboard URL (replaces wildcard `origins="\*"`)
- [] Set `SUPERADMIN\_PASSWORD` in `.env` (now required -- API will not start without it)

Dashboard Session Storage (fixed 2026-09-22)

The 2026-09-22 security audit flagged that dashboard tokens rode continuously in the URL (`?tok=/'&rtok=`) to survive Streamlit page reloads – landing in browser history and any server/proxy access log that records query strings. This is now fixed properly rather than just mitigated:

- New `dashboard/utils/session\_store.py` – the real access/refresh tokens live server-side in Redis (`dash:session:<id>`, TTL matching `JWT\_REFRESH\_TOKEN\_EXPIRES`), keyed by a random opaque session id (`secrets.token\_urlsafe(32)`). Only that id goes in the URL (`?sid=`) – a leaked/logged URL is now useless on its own.
- `dashboard/utils/auth.py::establish\_session()` is the single entry point every login flow (password, MFA) calls to create a session and stamp `?sid=`. `require\_auth()` keeps it in sync on reload; `api\_client.py`'s silent token-refresh path updates the stored tokens in place so a rotated access token survives the next reload too.
- Degrades gracefully if Redis is unreachable: falls back to the old raw-token-in-URL scheme (logged as a warning) rather than breaking login. Legacy `?tok=/'&rtok=` links (already-open tabs from before this change) are still accepted and migrated to a session on next load.
- Requires the dashboard process to reach the same Redis instance as the API (`REDIS\_URL` env var, same convention as `api/utils/cache.py`) – new `redis==5.2.1` dependency in `dashboard/requirements.txt`.

Known Residual Risk (audited 2026-09-22, not fixed)

- ****React frontend stores the JWT in `localStorage`**** (`frontend/src/contexts/AuthContext.tsx`, `frontend/src/api/client.ts`) – exfiltrable by any successful XSS on that page. ****Not fixed.**** The correct fix is httpOnly-cookie-issued JWTs with CSRF token handling, which changes how `api/routes/auth.py` issues tokens for every client (Streamlit dashboard included) – a cross-cutting auth redesign, tracked here as a named follow-up rather than attempted piecemeal.

12. Environment Variables Reference

Variable	Required	Default	Description
-----	-----	-----	-----
`DATABASE_URL`	✓	–	PostgreSQL connection string
`SECRET_KEY`	✓	–	Flask secret key (32-byte hex)
`JWT_SECRET_KEY`	✓	–	JWT signing key (32-byte hex)

```
| `ORG_REGISTRATION_TOKEN` | ✓ | - | Agent registration shared secret |
| `JWT_ACCESS_TOKEN_EXPIRES` | - | `900` | Access token TTL in seconds |
| `JWT_REFRESH_TOKEN_EXPIRES` | - | `604800` | Refresh token TTL in seconds |
| `REDIS_URL` | - | `redis://localhost:6379/0` | Redis connection URL |
| `CELERY_BROKER_URL` | - | `redis://localhost:6379/0` | Celery broker |
| `CELERY_RESULT_BACKEND` | - | `redis://localhost:6379/1` | Celery result backend |
| `API_BASE_URL` | - | `http://localhost:5000` | Used by dashboard to reach API |
| `SMTP_HOST` | - | - | SMTP server (omit to disable email alerts) |
| `SMTP_PORT` | - | `587` | SMTP port |
| `SMTP_USER` | - | - | SMTP login username |
| `SMTP_PASS` | - | - | SMTP login password (use app-specific password) |
| `SMTP_FROM` | - | `SMTP_USER` | From address for alert emails |
| `SUPERADMIN_EMAIL` | - | `superadmin@rmm.local` | Email for the auto-seeded superadmin
account |
| `SUPERADMIN_PASSWORD` | ✓ | - | **Now required at startup.** Min 10 chars. API raises
`RuntimeError` if missing or too short. |
| `CORS_ORIGINS` | - | `http://localhost:8501` | Comma-separated allowed CORS origins.
Set to dashboard URL(s) in production. |
| `DB_PASSWORD` | - | `changeme` | PostgreSQL password used in `docker-compose.yml` only.
|
| `FLASK_ENV` | - | `production` | Set to `development` for debug mode |
| `FLASK_DEBUG` | - | `0` | Set to `1` only in development |
| `SENTRY_DSN` | - | - | Sentry DSN. Omit to disable. No-op when unset.
`traces_sample_rate=0.05`, `send_default_pii=False` |
| `DASHBOARD_URL` | - | `http://localhost:3000` | React frontend URL – used in CORS and
Stripe redirect URLs |
| `MQTT_HOST` | - | - | MQTT broker hostname. Omit to disable MQTT ingestion. |
| `MQTT_PORT` | - | `1883` | MQTT broker port |
| `MQTT_USERNAME` | - | - | MQTT broker credentials (optional) |
| `MQTT_PASSWORD` | - | - | MQTT broker credentials (optional) |
| `MQTT_TOPIC_PREFIX` | - | `rmm` | Topic pattern: `{prefix}/{device_id}/sensors/{type}`
|
| `SNMP_TIMEOUT` | - | `3` | SNMP GET request timeout in seconds |
| `STRIPE_SECRET_KEY` | - | - | Stripe secret key for payment processing |
| `STRIPE_WEBHOOK_SECRET` | - | - | Stripe webhook signing secret |
| `LATEST_AGENT_VERSION` | - | `1.0.0` | Current agent version for auto-update checks |
| `AGENT_UPDATE_PATH` | - | - | URL path to download updated agent binary |
```

Generate secrets:

```
python -c "import secrets; print(secrets.token_hex(32))" # SECRET_KEY, JWT_SECRET_KEY
python -c "import secrets; print(secrets.token_hex(24))" # ORG_REGISTRATION_TOKEN
```

```
---

## 13. Docker Deployment

### docker-compose.yml (project root)

6 services:

| Service | Image | Command |
|-----|-----|-----|
```

```
| `db` | `postgres:16-alpine` | Postgres with health check |
| `redis` | `redis:7-alpine` | Redis with health check |
| `api` | Built from `api/Dockerfile` | `flask db upgrade && python app.py` |
| `celery_worker` | Same as `api` | `celery -A tasks.celery_app worker --pool=solo -l info` |
| `celery_beat` | Same as `api` | `celery -A tasks.celery_app beat -l info` |
| `dashboard` | Built from `dashboard/Dockerfile` | `streamlit run app.py` |
```

Dockerfiles

```
- `api/Dockerfile` — `python:3.11-slim` + `libpq-dev gcc` (for psycopg2 compile)
- `dashboard/Dockerfile` — `python:3.11-slim`
```

Usage

Start all services

`docker-compose up -d`

View API logs

`docker-compose logs -f api`

Stop (preserves data volume)

`docker-compose down`

Stop + delete all data

`docker-compose down -v`

Rebuild after code changes

`docker-compose build api dashboard && docker-compose up -d`

Environment note

When using Docker Compose, `DATABASE\_URL` must use `@db:5432` (service name), not `@localhost:5432`.
`REDIS\_URL` / `CELERY\_BROKER\_URL` must use `redis://redis:6379/...`.
`SUPERADMIN\_PASSWORD` is required — API container will exit immediately if missing.

Flask Reloader Stability (`use\_reloader=False`)

`app.run()` sets `use\_reloader=False` permanently:

```
app.run( host=os.getenv("API_HOST", "0.0.0.0"), port=int(os.getenv("API_PORT", 5000)),
debug=os.getenv("FLASK_DEBUG", "1") == "1", use_reloader=False, )
```

****Why:**** Flask's default ``watchdog`` reloader monitors all files reachable via ``sys.path``, including Redis/Memurai's site-packages (``resp2.py``, ``resp3.py``, etc.). Memurai writes to these files during normal operation, causing Flask to restart every ~30 seconds – killing in-flight network scan threads, resetting all connections, and returning 500/connection-reset errors to the dashboard.

``use_reloader=False`` eliminates these restarts. Code changes require a manual API restart during development. In production, this is the expected behavior regardless.

Security hardening applied at startup (``api/app.py``)

``_validate_env()`` runs at import time and raises ``RuntimeError`` if:

- ``SECRET_KEY`` or ``JWT_SECRET_KEY`` < 32 characters
- ``SUPERADMIN_PASSWORD`` unset or < 10 characters
- ``ORG_REGISTRATION_TOKEN`` is the placeholder value ``"replace-with-a-unique-org-token"``

Additional hardening:

- ``X-Request-ID`` header: ``before_request`` generates UUID if not present; echoed on all responses.
- Dev mode warning: API logs WARNING on startup if ``FLASK_ENV != "production"``.
- CORS restricted via ``CORS_ORIGINS`` env var (replaces ``origins="*"``).

14. AI Assistant

Architecture

[Streamlit page] ■ `render_ai_assistant(page_name, context)` ▼ [`dashboard/utils/ai_assistant.py`] ■ POST `/api/assistant/chat` (JWT Bearer) ▼ [`api/routes/assistant.py`] ■ `_build_system_prompt(role, page, context)` ■ `anthropic.Anthropic(api_key=...).messages.create(...)` ▼ [Anthropic API — `claude-haiku-4-5-20251001`] ■ reply text + suggested_actions list ▼ [Streamlit sidebar chat panel]

New files

File	Role
<code>`api/routes/assistant.py`</code>	Flask blueprint, chat endpoint, system prompt builder
<code>`dashboard/utils/ai_assistant.py`</code>	Streamlit sidebar widget

``POST /api/assistant/chat``

****Auth:**** JWT Bearer (same as all other API endpoints)

****Rate limit:**** 30 requests per minute per IP (Flask-Limiter)

****Request body:****

```
{ "message": "How do I deploy patches?", "page": "OS Patches", "context": { "pending_patches": "12",
"selected_device": "DESKTOP-01"}, "history": [ { "role": "user", "content": "..."}, { "role": "assistant", "content":
```

```
"..."]}] }
```

```
**Response:**
```

```
{ "reply": "To deploy patches: 1. Select a device...", "suggested_actions": ["Deploy OS patches", "Check patch status", "What devices need updates?"] }
```

```
**Constraints:**
```

- `message` max 1000 chars (truncated server-side)
- `history` capped at last 10 turns
- `context` keys/values sanitised: key max 60 chars, value max 300 chars
- `page` max 60 chars

```
**Error responses:**
```

Status	Condition
400	`message` empty
429	Rate limit exceeded
503	`AI_ASSISTANT_ENABLED=false`, or `ANTHROPIC_API_KEY` missing/invalid, or Anthropic API down

```
### System prompt structure
```

```
`_build_system_prompt(role, page, context)` assembles:
```

You are the AI Assistant embedded in Remote Management System (RMS)...

USER ROLE: {role} ROLE PERMISSIONS: {role_desc}

CURRENT PAGE: {page} PAGE PURPOSE: {page_desc}

LIVE CONTEXT:

- {key}: {value}

...

RESPONSE RULES:

1. Concise — 2-4 sentences for simple answers; numbered steps for how-to.
2. Use exact UI element names (button labels, tab names, headings).
3. Only describe features that exist. Never invent pages, buttons, or endpoints.
4. Respect role — if feature requires higher role, say so.
5. For viewers: never suggest create/edit/delete/run actions.
6. Off-topic: "I can only help with navigating this system."
7. Plain language — user may be new to RMM tools.
8. Multi-step tasks: number the steps.
9. If context shows a problem, mention the most urgent action first.

```
### Role extraction from JWT
```

```
from flask_jwt_extended import jwt_required, get_jwt
```

```
@assistant_bp.route("/chat", methods=["POST"]) @jwt_required() def chat(): role = get_jwt().get("role",
"viewer") # role is in additional_claims ...
```

```
> **Critical:** `get_jwt_identity()` returns the user ID (int), not the role. Role lives
in `additional_claims`. Always use `get_jwt()`.
```

```
### Dashboard widget
```

```
`render_ai_assistant(page_name, context)` in `dashboard/utils/ai_assistant.py`:
```

```
from utils.api_client import RMMClient
```

```
def render_ai_assistant(page_name="Overview", context=None): token =
st.session_state.get("access_token") if not token: return # not logged in — silently skip client =
RMMClient( access_token=token, refresh_token=st.session_state.get("refresh_token", ""), )
```

... initialise session state, render toggle, chat history, form, suggestions

```
> **Critical:** Do NOT use `st.session_state.get("_rmm_client")` — that key is never set.
`require_auth()` returns the client but does not store it. Build `RMMClient` directly
from `access_token`.
```

```
**Session state keys:**
```

Key	Type	Purpose
<code>_ai_history</code>	<code>list[dict]</code>	Chat turns <code>{role, content}</code> — capped display at 14
<code>_ai_open</code>	<code>bool</code>	Panel open/closed
<code>_ai_seen_onboard</code>	<code>bool</code>	Whether welcome message has been shown this session
<code>_ai_suggested</code>	<code>list[str]</code>	Quick action button labels from last API response

```
**Page key:** widget keys are scoped to page_name.lower().replace(" ", "_").replace("/",
"_")` to prevent duplicate-key errors across pages.
```

```
### Adding the widget to a page
```

```
from utils.ai_assistant import render_ai_assistant
```

At the bottom of the page (after all page content):

```
render_ai_assistant("Page Name")
```

With live context (data-heavy pages):

```
render_ai_assistant("Overview", { "total_devices": total, "online_devices": online, "open_alerts": alerts, })
```

The widget renders inside ``with st.sidebar:`` – no layout columns needed.

Environment variables

Variable	Required	Default	Description
<code>`ANTHROPIC_API_KEY`</code>	Yes	–	Anthropic API key (<code>`sk-ant-api03-...`</code>)
<code>`AI_ASSISTANT_MODEL`</code>	No	<code>`claude-haiku-4-5-20251001`</code>	Model ID
<code>`AI_ASSISTANT_ENABLED`</code>	No	<code>`true`</code>	Set <code>`false`</code> to disable without removing code

Dependency

api/requirements.txt

```
anthropic>=0.40.0
```

Install: ``pip install anthropic>=0.40.0``

Security properties

- API key stored server-side only (``api/.env``) – never sent to dashboard or browser
- JWT required on ``/api/assistant/chat`` – unauthenticated requests return 401
- Rate limiting prevents API cost abuse (30 req/min per IP via Flask-Limiter)
- Conversation history stored in Streamlit session state only – cleared on logout/refresh, never persisted to DB
- Context values sanitised before inclusion in system prompt

Extension points

****Add a new page description:**** Add an entry to ``_PAGE_INFO`` in ``api/routes/assistant.py``. Key must match the ``page_name`` string passed from the dashboard page.

****Add quick actions for a page:**** Add an entry to ``_PAGE_ACTIONS`` with the same key.

****Add role-aware behaviour:**** Update ``_ROLE_CAPABILITIES`` dict.

*****"Do it for me" actions (future):**** The widget already holds a ``RMMClient`` instance. Wire suggested-action buttons to call ``client.create_ticket(...)`` etc. instead of just sending a chat message.

****Voice input (future):**** Add ``st.audio_input()`` → transcribe → feed transcript to ``_send_message()``. No backend changes needed.

- 400/422 error handlers: detail logged server-side only, generic message returned to client.

16. Commercial Audit – Phase F

F.1 Agent Multi-Customer Routing

`api/schemas/agents.py` – `customer\_id` = fields.String(load_default="", validate=validate.Length(max=36))`

`api/routes/agents.py` registration – if `customer\_id` in payload and valid UUID: assigns agent to that customer. Otherwise falls back to
`Customer.query.filter\_by(is\_active=True).first()` (backward compatible).

`agent/config.ini` – `customer\_id` = under `[api]`. Set via `setup\_agent.py <server\_ip> <org\_token> <customer\_id>`.

F.2 DPAPI Agent Token Encryption

`agent/rmm\_agent.py` – `\_protect\_token()` / `\_unprotect\_token()`. Windows: encrypts with `win32crypt.CryptProtectData`, stores as `DPAPI:<base64>`. Non-Windows or error: stores plaintext. Existing plaintext tokens auto-migrate on first write. Requires `pywin32` on Windows.

F.3 OpenAPI 3.0 / Swagger

`GET /api/docs` – SwaggerUI (CDN, brand topbar). `GET /api/openapi.json` – raw spec (47 paths, 16 tags, BearerAuth). No new pip deps.

F.4 Redis Cache (`api/utils/cache.py`)

`cache\_get(key)` / `cache\_set(key, value, ttl)` / `cache\_delete(key)`. JSON-serialized Redis values with TTL. Silent no-op on Redis unavailability.

Applied: dashboard summary (30s, per-tenant key), platform_counts (60s).

F.5 Alert Thundering Herd Fix

Redis `SET NX EX 55` lock in `evaluate\_all\_rules`. Lock key:
`rmm:lock:evaluate\_all\_rules`. If lock not acquired, task exits. Released in `finally`.

F.6 Webhook Dispatch (`api/utils/webhook.py`)

`dispatch\_alert\_webhooks(channels, rule\_name, device\_hostname, message, severity)`.

Formats: Slack Block Kit (color-coded), MS Teams MessageCard, generic JSON POST. 5s timeout. Never raises.

Terminal Improvements

| Fix | Detail |

```
|-----|-----|
| Output polling | `TerminalPage.tsx` polls `GET
/terminal/sessions/<id>/output?after=<last_id>` every 2s instead of reading from the POST
/commands response (which only returns a pending status object, not output). |
| Pending-command indicator | `GET /terminal/sessions/<id>/output` now returns
`pending_commands` count. Dashboard shows amber blinking indicator when commands are
queued but agent has not yet picked them up. |
| Alert storm fix | `evaluate_all_rules` uses separate `open_alert_map` (no time filter)
and `resolved_alert_map` (cooldown after recovery) – prevents indefinite re-fire of open
alerts that aged past `cooldown_minutes`. |
| Token rotation | `terminal_worker.update_token()` called from `rmm_agent` on token
rotation – terminal worker no longer continues with a revoked token. |
| Session seq reset | Auto-close fragment resets `_term_output_seq` to 0 on session close
– prevents stale buffer on next connect. |
```

F.7 Device Ownership (Client Role)

```
- `POST /api/terminal/sessions` – returns 404 if `device.customer_id !=
claims["customer_id"]`
- `POST /api/scripts/<id>/run` – filters `device_ids` to customer-scoped query;
unauthorized IDs skipped
```

F.8 Script Run Audit

```
`run_script()` emits `AuditLog(action="script_run", resource_type="script",
payload={script_name, device_count, device_ids})` after commit.
```

F.9 GDPR Endpoints

```
| Endpoint | Behaviour |
|-----|-----|
| `GET /api/admin/users/<id>/gdpr-export` | Returns `{user, audit_log[1000],
ticket_comments[1000]}` |
| `DELETE /api/admin/users/<id>/gdpr-delete` | Anonymizes PII, scrubs IPs from audit log,
nulls comment author email. Irreversible. |
```

Both admin-only. Both emit `AuditLog` entry.

F.10 Billing Automation

```
`Customer` gains `billing_day`, `per_device_rate`, `tax_rate` (migration `j1k2l3m4n5o6`).
```

```
`tasks/billing_tasks.py` `generate_recurring_invoices` (beat 86400s): generates draft
invoice for prior month for customers where `billing_day == today` and `per_device_rate >
0`. Idempotent on `(customer_id, period_start)`.
```

F.11 Configurable SLA Policies

```
`SLAPolicy` model: `(customer_id nullable, priority)` unique. Seeded with 4 global
defaults.
```

```
Routes at `/api/sla-policies/`. `tickets.py` `_sla_resolution_hours()` lookup:
customer-specific → global → hardcoded.
```

```
### F.12 Database Backup (`tasks/backup_tasks.py`)
```

```
`backup_database` (beat 86400s): pg_dump → gzip → `BACKUP_DIR`. Prunes files older than  
`BACKUP_RETAIN_DAYS` days. Retry on timeout (3600s) or error (1800s).
```

```
Env: `BACKUP_DIR` (default `../backups`), `BACKUP_RETAIN_DAYS` (default 7).
```

```
Restore: `gunzip -c backups/rmmdb_YYYYMMDD_HHMMSS.sql.gz | psql -U rmm_app rmmdb`
```

```
### F.13 Migration Chain (complete)
```

```
d746c05420bd → 2c8f1d3e9a4b → 93baa3927b0c → f3e2d1c0b9a8 → a1b2c3d4e5f6 → e5d4c3b2a1f0  
→ b5c4d3e2f1a0 → c1d2e3f4a5b6 → d2e3f4a5b6c7 → e6f7a8b9c0d1 → f1a2b3c4d5e6 → a2b3c4d5e6f7  
→ b3c4d5e6f7a8 → c4d5e6f7a8b9 → d5e6f7a8b9c0 → f7a8b9c0d1e2 → h9i0j1k2l3m4 → i0j1k2l3m4n5  
→ j1k2l3m4n5o6 → k2l3m4n5o6p7 → l3m4n5o6p7q8 (IoT sensor_readings table)
```

```
---
```

```
## 15. React Frontend
```

```
### Stack
```

Package	Version	Role
React	19	UI framework
Vite	8	Dev server + build tool
TypeScript	6	Type safety
TanStack Query	5	Server state + caching
React Router	7	Client-side routing
Tailwind CSS	3	Utility-first styling
axios	1.x	HTTP client
lucide-react	1.x	Icon library

```
### Pages
```

```
All 19 pages match Streamlit equivalents: `DashboardPage`, `TicketsPage`,  
`CustomersPage`, `DevicesPage`, `AlertsPage`, `AdminPage`, `AutomationPage`,  
`BillingPage`, `ClientPortalPage`, `DiskManagementPage`, `MaintenancePage`,  
`NetworkPage`, `OSPatchesPage`, `ProfilePage`, `ReportsPage`, `ScriptsPage`,  
`SoftwarePatchesPage`, `TerminalPage` (with output polling fix applied), `auth/` (login  
flow).
```

```
### Running
```

```
cd frontend npm install npm run dev # http://localhost:3000 npm run build # production build →  
frontend/dist/ npm run lint # oxlint
```

```
### CORS configuration
```

```
Set `CORS_ORIGINS=http://localhost:3000` in `api/.env`. For production, include both the  
Streamlit and React URLs:
```

CORS_ORIGINS=http://localhost:8501,https://dashboard.example.com

Auth flow

Same JWT Bearer token flow as Streamlit. Tokens stored in `localStorage` (React) vs `st.session\_state` + URL params (Streamlit). The React frontend uses `AuthContext` to hold `access\_token` and `refresh\_token`; axios interceptors handle 401 → refresh → retry automatically.

16. Screenshot Capture Pipeline

Agent side (`agent/screenshot.py`)

`capture()` returns JPEG bytes or `None`:

- **Windows/macOS:** `PIL.ImageGrab.grab()` (Pillow)
- **Linux:** `scrot` subprocess → `gnome-screenshot` → PIL fallback
- Resizes to max 1920px wide, JPEG quality 72 (~100-400 KB per capture)
- Returns `None` silently on headless/no-display – non-fatal

Periodic sync in `rmm\_agent.py`:

```
screenshot_interval = config.getint("agent", "screenshot_interval", fallback=300)
```

All errors caught at DEBUG level – never breaks the heartbeat loop.

API endpoints

Method	Path	Auth	Description
POST	/api/agents/<device_id>/screenshot	X-Agent-Token	Accepts `image/jpeg` or `image/png`, 5 MB cap. Saves to `api/screenshots/<device_id>.jpg`. Replaces on each upload.
GET	/api/devices/<device_id>/screenshot	JWT	Returns latest screenshot via `send_file`. 404 JSON error if no screenshot captured yet.

Screenshots are stored in `api/screenshots/` (gitignored, `.gitkeep` present). One file per device – only the latest capture is retained. The directory is not served via HTTP; the API streams the file through `send\_file`.

Dashboard

The Devices page (agent device rows) shows the latest screenshot inline when available. The React `DevicesPage` fetches `GET /api/devices/<id>/screenshot` and renders it as an `` tag.

17. IoT / MQTT / SNMP

IoT Sensor Agent (`agent/iot\_agent.py`)

Lightweight sensor collector for Raspberry Pi / Linux SBC. Reuses `agent/config.ini` and

```
`heartbeat.py` APIClient for auth.
```

Sensor backends (all optional – skipped if library not installed):

Backend	Library	Sensors
-----	-----	-----
`/sys/class/hwmon`	none	CPU/board temperature
DHT11/DHT22	`adafruit-dht`	Temperature + humidity
BME680	`adafruit-circuitpython-bme680`	Temp, humidity, pressure, gas
PIR motion	`gpiozero`	Motion detection
Door reed switch	`gpiozero`	Open/closed state

Config in `agent/config.ini` under `[iot]`:

```
[iot] interval = 60 dht_type = DHT22 dht_pin = 4 bme680_i2c_addr = 0x77 pir_pin = 17 door_pin = 27
```

Setup: `python setup\_agent.py <server\_ip> <org\_token>` (same as standard agent), then
`python iot\_agent.py`.

API: Sensor Readings (`api/routes/sensors.py`)

Method	Path	Auth	Description
-----	-----	-----	-----
POST	`/api/sensors/<device_id>/readings`	X-Agent-Token	Ingest sensor readings. Body: `[{sensor_type, value, unit, channel}]`
GET	`/api/sensors/<device_id>/readings`	JWT	List readings for device (filterable by sensor_type, time range)
GET	`/api/sensors/<device_id>/latest`	JWT	Latest reading per sensor_type

MQTT Ingestion (`api/tasks/mqtt\_tasks.py`)

Celery task subscribes to `{MQTT\_TOPIC\_PREFIX}/#` when `MQTT\_HOST` is set. Topic convention: `{prefix}/{device\_id}/sensors/{sensor\_type}`. Payload: `{"value": <float>, "unit": "<str>", "channel": "<str|null>"}`. Writes to `SensorReading` via the same path as the direct agent POST.

No-op when `MQTT\_HOST` is blank – no errors raised.

SNMP Polling (`api/tasks/snmp\_tasks.py`)

Per-device SNMP polling task. Community string stored in
`Device.metadata.snmp\_community` (set via device Edit form in Admin). `SNMP\_TIMEOUT` env
var controls per-request timeout (default 3s).

Dashboard: IoT Sensors page (`dashboard/pages/18\_IoT\_Sensors.py`)

Shows sensor readings as time-series charts per sensor type. Displays latest value, unit,
and channel for each sensor. Admin/Tech access only.

18. Test Suite

102 tests across 6 files, all passing.

```

| File | Tests | Coverage |
|-----|-----|-----|
| `api/tests/test_auth.py` | Auth flows, MFA, force-change-password, rate limits | JWT, bcrypt, TOTP |
| `api/tests/test_agents.py` | Registration, heartbeat, task result, patch/software submit | Agent token, device upsert |
| `api/tests/test_alerts.py` | Alert CRUD, acknowledge, resolve, auto-resolve on recovery | Rule evaluation logic |
| `api/tests/test_tickets.py` | Ticket CRUD, comments, SLA due-date calc, GDPR export/delete | Role guards, SLA policy lookup |
| `api/tests/test_devices.py` | Device list/get/update/delete, metrics, screenshot endpoint | Pagination, platform_counts |
| `api/tests/test_cache.py` | `cache_get`/`cache_set`/`cache_delete`, TTL expiry, Redis unavailability no-op | Redis mock |

### Running tests

```

```
cd api .\venv\Scripts\Activate.ps1 pytest tests/ -v --cov=. --cov-report=term-missing
```

Requires a running PostgreSQL + Redis (or the CI services defined in ``.github/workflows/ci.yml``). ``conftest.py`` force-sets ``ORG_REGISTRATION_TOKEN`` and creates an in-memory test app via ``create_app("testing")``.

CI pipeline (``.github/workflows/ci.yml``)

- PostgreSQL 15 + Redis 7 service containers
- ``flask db upgrade`` with ``FLASK_ENV=development``
- ``pytest`` with coverage upload to Codecov
- TypeScript type-check (``tsc --noEmit``) on ``frontend/``
- Docker build gate on ``main`` branch pushes

19. Mobile Device Management

Overview

Real management (lock, wipe, lost-mode, compliance reporting) for phones that cannot run the RMM's Python agent. No custom agent binary is written or shipped – Android devices run Google's own signed "Android Device Policy" app, provisioned during a device-owner-initiated enrollment (QR code or link, never silent). The API talks to Google's Android Management API server-side; the phone never talks to this RMM directly. iOS is not implemented – see [iOS status](#ios-status) below.

Key files: ``api/models/mdm_integration.py``, ``api/utils/android_mgmt.py``, ``api/routes/mobile_mdm.py``, ``api/tasks/mdm_tasks.py``, ``api/utils/crypto.py``, ``dashboard/pages/19_Mobile_Enrollment.py``.

Data model

`**`MdmIntegration`**` (``mdm_integrations``) – one row per bound Android Enterprise:

Field	Notes
`customer_id`	Nullable – null means staff-wide, set means a dedicated per-customer binding
`project_id` / `enterprise_id`	Google Cloud project + `enterprises/{id}` resource name (set once binding completes)
`service_account_json_enc`	Fernet-encrypted (`api/utils/crypto.py`), never written to disk
`apple_push_cert_enc` / `apple_topic`	Reserved placeholders for a future iOS phase – unused today

MobileEnrollment (`mobile\_enrollments`) – one-to-zero-or-one with `Device`:

Field	Notes
`device_id`	Nullable until Google confirms enrollment, unique
`ownership_type`	`byod` \ `corporate` – maps to Android's `allowPersonalUsage`
`status`	`pending` \ `enrolled` \ `revoked` \ `wiped` \ `expired`
`consent_given_by_user_id`, `consent_given_at`, `consent_text_version`, `consent_ip_address`	Real columns, not a JSON blob – this is the legal consent record

`Device.is\_agentless` stays `True` for managed phones – they still run no RMM agent. Real action buttons in the dashboard gate on `device.mdm\_enrollment.status == "enrolled"`, not `is\_agentless`.

API: `/api/mdm` (`api/routes/mobile\_mdm.py`)

Method	Path	Auth	Description
POST	/api/mdm/integrations	admin	Create integration stub
POST	/api/mdm/integrations/<id>/credentials	admin	Multipart upload of the Google service-account JSON
POST	/api/mdm/integrations/<id>/bind	admin	Step 1 of enterprise binding – returns `{signup_url}`
GET	/api/mdm/integrations/<id>/bind_callback	none (state-nonce validated)	Google's browser redirect target
PUT	/api/mdm/integrations/<id>/policy	admin, technician	Push a policy JSON to the bound enterprise
GET	/api/mdm/available_integrations	any authenticated role	Secret-free list scoped to the caller's customer – powers the Client Portal's self-enrollment dropdown
POST	/api/mdm/enrollments	admin, technician, client	Create an enrollment token; `role=client` is server-forced to `ownership_type=byod` and requires `consent_acknowledged=true`
DELETE	/api/mdm/enrollments/<id>	scope-checked	Revoke – `RELINQUISH_OWNERSHIP` for BYOD, full removal for corporate
POST	/api/mdm/devices/<id>/{lock,reboot,reset_password,wipe,start_lost_mode,stop_lost_mode}	scope-checked, per-command role	Issues an `issueCommand` – applies on the device's next check-in, not instantly

`wipe` is `admin`-only. `lock`/`start\_lost\_mode`/`stop\_lost\_mode` are the only commands a `role=client` may issue, and only on their own scoped device.

Practical example: enroll and lock a company phone

1. Create the integration (once, per Google Cloud project)

```
curl -X POST http://localhost:5000/api/mdm/integrations \ -H "Authorization: Bearer $ADMIN_JWT" -H "Content-Type: application/json" \ -d '{"name": "Acme Fleet", "project_id": "acme-rmm-mdm"}'
```

→ `{"id": "e1f2...", "type": "android", "bound": false, ...}`

2. Upload the service-account JSON, then bind (opens Google's own signup page)

```
curl -X POST http://localhost:5000/api/mdm/integrations/e1f2.../credentials \ -H "Authorization: Bearer $ADMIN_JWT" -F "file=@service-account.json" curl -X POST http://localhost:5000/api/mdm/integrations/e1f2.../bind \ -H "Authorization: Bearer $ADMIN_JWT" -H "Content-Type: application/json" \ -d '{"callback_base_url": "https://your-tunnel.example.com"}'
```

→ `{"signup_url": "https://enterprise.google.com/signup/android/...&state=..."}`

Admin opens `signup_url` in a browser, completes Google's hosted flow,

Google redirects to `bind_callback` — `integration.enterprise_id` is now set.

3. Create a corporate enrollment and get the QR/link

```
curl -X POST http://localhost:5000/api/mdm/enrollments \ -H "Authorization: Bearer $ADMIN_JWT" -H "Content-Type: application/json" \ -d '{"mdm_integration_id": "e1f2...", "ownership_type": "corporate"}'
```

→ {

"enrollment_id": "9a8b...",

"qr_code_json": "{...}", # render with the `qrcode` package

"enrollment_link": "https://enterprise.google.com/android/enroll?et=ABC123",

"expires_at": "2026-09-22T16:30:00+00:00"

}

Technician scans this at a factory-reset phone's welcome screen.

The next 5-minute beat tick
(`tasks.mdm_tasks.sync_all_mdm_integrations`)

links the enrollment to a new Device row automatically.

4. Once enrolled, issue a real command

```
curl -X POST http://localhost:5000/api/mdm/devices/<device_id>/lock \ -H "Authorization: Bearer $ADMIN_JWT"
```

→ 202 {"message": "Command sent — applies when the device next checks in."}

Practical example: minimal policy JSON

Pushed via ``PUT /api/mdm/integrations/<id>/policy``, stored as ``MdmIntegration.default_policy_name`` and applied to every device enrolled under it:

```
{ "policy_name": "default", "policy": { "passwordRequirements": { "passwordMinimumLength": 6 },  
  "cameraDisabled": false, "applications": [ { "packageName": "com.slack", "installType":
```

```
"FORCE_INSTALLED"}}}]}
```

App push/removal, kiosk mode, and per-app VPN are all additional fields on this same ``policy`` object – no new endpoint or table needed to extend it (see Google's Android Management API ``policies`` reference for the full schema, which evolves – re-verify field names before relying on any not shown here).

```
### Celery sync (`tasks.mdm_tasks`)
```

``sync_all_mdm_integrations`` runs every 5 minutes (``celery_app.py`` beat schedule), fanning out ``sync_mdm_integration.delay(id)`` per active, bound integration. Each sync call: lists devices via ``enterprises.devices.list``, matches new devices to still-pending ``MobileEnrollment`` rows, creates the corresponding ``Device`` row on first sight, and updates ``policy_compliant`/`hardware`` fields on every pass – the same shape as ``tasks.psa_tasks.sync_all_psa_integrations``.

```
### Consent enforcement
```

Enrollment consent is checked server-side, not just hidden in the UI: ``create_enrollment`` in ``mobile_mdm.py`` forces ``ownership_type="byod"`` for any ``role=client`` caller regardless of the request body, requires ``consent_acknowledged=true`` for BYOD, and 404s if a client tries to enroll into an integration scoped to a different customer's ``customer_id``. Every consent event and every issued command writes an ``AuditLog`` row (``resource_type="mobile_device"``).

```
### iOS status
```

Not implemented. Real Apple MDM requires an APNs push certificate, which Apple only counter-signs for organizations enrolled in Apple Business Manager or for already vendor-signed MDM servers (Fleet, MicroMDM/NanoMDM) – a business/account decision outside this codebase. ``MdmIntegration.type`` already accepts ``"apple"`` and the ``apple_push_cert_enc`/`apple_topic`` columns exist, so this phase needs no schema migration when the decision is made – only a new branch in ``MdmIntegration.get_client()``, which currently raises ``NotImplementedError`` for ``type="apple"``.

```
## 20. API & Token Usage Monitoring
```

```
### Overview
```

Superadmin-only backend auditing/reporting for API call volume and AI token consumption, built after usage got unexpectedly high with no visibility into which service, feature, or user was responsible. Strictly gated to ``role == "superadmin"`` – ``admin`` gets 403/sees nothing, unlike every other admin surface in this app.

Key files: ``api/models/usage.py``, ``api/utils/usage_tracker.py``, ``api/routes/usage.py``, ``api/tasks/usage_tasks.py``, ``dashboard/pages/22_Usage_Monitoring.py``, ``frontend/src/pages/UsageMonitoringPage.tsx``.

```
### Data model
```

`**`ApiUsageEvent`**` (``api_usage_events``) – one row per instrumented call:

Field	Notes
-----	-----

```
| `service` | `ai_assistant`, `stripe`, `psa_connectwise`, `psa_autotask`, `android_mdm`,
`email_imap`, `network_scan`, `webhook_slack`/`webhook_teams`/`webhook_generic`, `smtp` |
| `feature` | Sub-label – dashboard page name for `ai_assistant`, action name (e.g.
`get_companies`) for integrations |
| `user_id` | Nullable – null for Celery/system-triggered calls |
| `input_tokens` / `output_tokens` / `estimated_cost_usd` | Only populated for
`ai_assistant`; cost is a configurable estimate, not billing data |
| `status` / `status_code` / `latency_ms` / `error_message` | `error_message` is a
truncated exception type/message only – never a raw response body, header, or secret |
```

ApiUsageHourly (`api\_usage\_hourly`) – durable rollup of internal Flask API request volume (`bucket\_start`, `endpoint`, `method`, `request\_count`, `error\_count`, `total\_latency\_ms`). Too high-volume to log one `ApiUsageEvent` row per request, so `utils/usage\_tracker.py::record\_internal\_api\_call` increments a Redis hash (`rmm:usage:hourly:{YYYYMMDDHH}`, 8-day TTL) on every request via `app.py`'s existing `\_log\_request` `after\_request` hook, and `tasks.usage\_tasks.persist\_hourly\_usage\_rollup` (hourly beat) upserts the previous completed hour's counters into this table before the Redis TTL expires.

UsageAlertConfig (`usage\_alert\_config`) – singleton row (`id="default"`):
`is\_enabled`, `spike\_multiplier` (default 3.0), `notification\_channels` (same JSON shape as `AlertRule.notification\_channels`). Deliberately its **own** table, not an `AlertRule` row – an `AlertRule` row would be visible to `admin`/`technician` via the Alerts page's "Alert Rules" tab, leaking this feature's existence/config to roles that must not see it.

Migration: `q8r9s0t1u2v3` (revises `p7q8r9s0t1u2`).

API: `/api/admin/usage` (`api/routes/usage.py`)

Method	Path	Auth	Description
GET	/api/admin/usage/summary?range=today\ 7d\ 30d`	superadmin only	Per-service totals (calls, tokens, estimated cost, error rate) + live anomaly flags
GET	/api/admin/usage/timeseries?range=&service=&metric=calls\ tokens\ cost`	superadmin only	Bucketed series for charts
GET	/api/admin/usage/by-feature?range=&service=`	superadmin only	Top features/pages/users breakdown
GET	/api/admin/usage/events?service=&status=&page=`	superadmin only	Paginated raw event drill-down
GET / PUT	/api/admin/usage/alert-config`	superadmin only	Read/update the spike-alert config

`\_require\_superadmin()` in this file has **no** admin bypass – unlike every other route file's `\_require\_role()`, which lets `superadmin` bypass an explicit admin/technician list. Here even `admin` is excluded outright.

Practical example: check today's AI Assistant spend

```
curl http://localhost:5000/api/admin/usage/summary?range=today\ -H "Authorization: Bearer $SUPERADMIN_JWT"
```

→ {

"range": "today",

"services": [

{"service": "ai_assistant", "calls": 214, "input_tokens": 61000,

"output_tokens": 38500, "estimated_cost_usd": 0.2535, "error_count": 1, "error_rate": 0.005},

{"service": "internal_api", "calls": 18420, "input_tokens": 0, "output_tokens": 0,

"estimated_cost_usd": 0, "error_count": 32, "error_rate": 0.002}

],

"totals": {"calls": 18634, "tokens": 99500, "estimated_cost_usd": 0.2535,

"error_count": 33, "error_rate": 0.0018},

"anomalies": []

}
